

Trabajo de Fin de Grado

Framework Backend–Frontend para el desarrollo rápido de proyectos apoyado en un modelo de datos en evolución: aplicación a los proyectos con el Jardín Botánico Viera y Clavijo

TITULACIÓN: Grado en Ingeniería Informática

AUTOR: Nicolás Rey Alonso

TUTORIZADO POR:
Rafael Juan Nebot Medina
María Dolores Afonso Suárez

Febrero de 2026

SOLICITUD DE DEFENSA DE TRABAJO DE FIN DE TÍTULO

D./D^a Nicolás Rey Alonso, autor del trabajo Framework Backend–Frontend para el desarrollo rápido de proyectos apoyado en un modelo de datos en evolución: aplicación a los proyectos con el Jardín Botánico Viera y Clavijo, correspondiente a la titulación Grado de Ingeniería Informática, en colaboración con la empresa ITC (Instituto Tecnológico de Canarias).

SOLICITA

que se inicie el procedimiento de defensa del mismo, para lo que se adjunta la documentación requerida, haciendo constar que

☒ se autoriza / ☐ no se autoriza la grabación en audio de la exposición y turno de preguntas.

Asimismo, con respecto al registro de la propiedad intelectual/industrial del TFT, declara que:

☐ Se ha iniciado o hay intención de iniciarlo (defensa no pública).

☐ No está previsto.

Y para que así conste firma la presente. (fecha en firma electrónica)

El/La estudiante

Fdo. _____

A rellenar y firmar **obligatoriamente** por el/la/los/las tutores

En relación con la presente solicitud, se informa: (firmar donde corresponda)

Positivamente (en caso de detección de copia, esta firma quedará invalidada)

Fdo. _____

Negativamente (justificación en TFT05)

Fdo. _____

DIRECTOR DE LA ESCUELA DE INGENIERÍA INFORMÁTICA

Agradecimientos

A Rafael Nebot y al ITC por concederme la oportunidad de participar en este proyecto.

A Marilola Afonso por el apoyo en el desarrollo de este proyecto.

A María Soledad por haberme hecho hincar los codos.

A María Teresa y a José Manuel por haberme apoyado.

A Manuel Jesús por aguantarme y ser la víctima de mis incontables ensayos.

A Brandon Guardia por estar presente en el desarrollo de este proyecto.

Resumen

El desarrollo de aplicaciones web de gestión de datos en entornos de investigación se enfrenta a una tensión recurrente: la necesidad de entregar software de calidad en plazos cortos frente a la inevitable evolución del modelo de datos a lo largo del proyecto. Cada vez que el modelo cambia, el coste de mantener sincronizados el backend, la capa de comunicación y la interfaz de usuario crece de forma desproporcionada, ya que buena parte de ese código se escribe a mano y se acopla a un dominio concreto.

Este Trabajo de Fin de Grado aborda dicho problema mediante el diseño, refactorización y consolidación de un *framework full-stack* reutilizable, compuesto por un backend en Python (*uniback*, basado en FastAPI, SQLAlchemy 2.x y PostgreSQL) y una librería de componentes Angular (*ngt-gui*). El núcleo de la contribución es una cadena de generación automática que parte del modelo ORM, deriva un *JSON Schema* enriquecido con pistas de presentación y, a partir de él, construye formularios dinámicos con Formly sin escribir HTML específico de cada entidad. El trabajo se estructura en una metodología iterativa incremental y se valida sobre las necesidades reales de los proyectos del Instituto Tecnológico de Canarias con el Jardín Botánico Viera y Clavijo.

Los resultados muestran un contrato de comunicación uniforme entre backend y frontend, un sistema de descubrimiento de entidades, una capa cliente genérica tipada y un generador de formularios guiado por el esquema, todo ello respaldado por pruebas automatizadas. El framework reduce drásticamente el código repetitivo necesario para exponer y editar una entidad nueva, y queda preparado para su reutilización en proyectos con modelos de datos en evolución.

Palabras clave: framework, generación automática de formularios, JSON Schema, FastAPI, Angular, Formly, modelo de datos en evolución, desarrollo rápido de aplicaciones.

Abstract

Building data-management web applications in research environments faces a recurring tension: delivering quality software on short schedules versus the inevitable evolution of the data model throughout the project. Every time the model changes, the cost of keeping the backend, the communication layer and the user interface in sync grows disproportionately, because much of that code is hand-written and coupled to a specific domain.

This Bachelor's Thesis tackles that problem through the design, refactoring and consolidation of a reusable full-stack framework, composed of a Python backend (*uniback*, based on FastAPI, SQLAlchemy 2.x and PostgreSQL) and an Angular component library (*ngt-gui*). The core contribution is an automatic generation pipeline that starts from the ORM model, derives an enriched JSON Schema with presentation hints and, from it, builds dynamic Formly forms without writing entity-specific HTML. The work follows an iterative and incremental methodology and is validated against the real needs of the projects carried out by the Instituto Tecnológico de Canarias with the Jardín Botánico Viera y Clavijo.

The results include a uniform communication contract between backend and frontend, an entity discovery system, a generic typed client layer and a schema-driven form generator, all backed by automated tests. The framework sharply reduces the boilerplate required to expose and edit a new entity, and is prepared for reuse in projects with evolving data models.

Keywords: framework, automatic form generation, JSON Schema, FastAPI, Angular, Formly, evolving data model, rapid application development.

Índice general

Listings	VII
Algoritmos	VIII
Figuras	IX
Cuadros	X
1. Introducción	1
1.1. Contexto	1
1.2. Descripción del problema	2
1.3. Solución propuesta	2
1.4. Estructura del documento	2
2. Estado actual y objetivos iniciales	4
2.1. Motivación y antecedentes	4
2.1.1. El ecosistema previo: NextGenDem, uniback y ngt-gui	4
2.1.2. La colaboración con el Jardín Botánico Viera y Clavijo	5
2.2. Estado del desarrollo	5
2.3. Planificación previa al desarrollo	5
2.3.1. Análisis previo	6
2.3.2. Acuerdo de planificación evolutiva	6
2.3.3. Tareas iniciales	6
2.4. Objetivos	6
3. Aportaciones del trabajo	8
3.1. Principales aportaciones	8
3.2. Competencias específicas	8
3.3. Alineamiento con los objetivos de desarrollo sostenible	9
4. Marco tecnológico y estado del arte	11
4.1. Tecnologías del backend	11
4.1.1. FastAPI y Pydantic	11
4.1.2. SQLAlchemy 2.x y Alembic	11
4.1.3. SQLAlchemy-Continuum	11

4.1.4. Infraestructura: PostgreSQL, Docker, Redis y Celery	12
4.2. Tecnologías del frontend	12
4.2.1. Angular	12
4.2.2. Formly	12
4.3. Estado del arte: generación de interfaces dirigida por esquema	12
5. Análisis de requisitos	14
5.1. Identificación de <i>stakeholders</i>	14
5.2. Requisitos funcionales	14
5.3. Requisitos no funcionales	16
5.4. Casos de uso	16
5.5. Restricciones	18
6. Diseño y arquitectura	19
6.1. Visión general	19
6.2. Arquitectura del backend (<i>uniback</i>)	20
6.3. Sistema de plugins y extensibilidad	20
6.3.1. El contrato del plugin	20
6.3.2. Descubrimiento e integración en el inicializador	21
6.3.3. Extensiones inyectables y registros	22
6.3.4. Ejemplo: el plugin de plantas	22
6.4. Arquitectura del frontend (<i>ngt-gui</i>)	23
6.4.1. Catálogo de modelos del núcleo	23
6.4.2. Catálogo de la librería de componentes	24
6.5. Persistencia, sesiones y configuración	24
6.6. Modelo polimórfico de objetos funcionales	25
6.7. Modelo de autorización	25
6.8. El contrato como frontera de desacoplamiento	25
6.9. Escenario de integración extremo a extremo	26
7. Desarrollo	27
7.1. Metodología	27
7.2. Planificación	27
7.3. Fases de desarrollo	28
7.3.1. Iteración 1	28
7.3.2. Iteración 2: contrato de comunicación back–front	32
7.3.3. Iteración 3: generación dinámica de formularios con Formly	35
7.3.4. Iteración 4: generación automática de esquemas	38
7.4. Pruebas	40
7.4.1. Backend	40
7.4.2. Frontend	40
7.4.3. Resumen de cobertura	41
8. Resultados	42
8.1. Resultados por incremento	42

8.1.1. Incremento 1: API backend y modelado (consolidado)	42
8.1.2. Incremento 2: contrato back-front (consolidado)	42
8.1.3. Incremento 3: formularios dinámicos (consolidado)	43
8.1.4. Incremento 4: bundle de esquemas (consolidado)	43
8.2. Valoración global	43
8.2.1. Comparativa antes/después	43
9. Conclusiones y trabajo futuro	45
9.1. Conclusiones	45
9.2. Trabajo futuro	45
9.3. Uso de la IA	46
10. Aspectos económicos y temporales	47
10.1. Planificación temporal	47
10.2. Presupuesto	47
11. Normativa y legislación	49
11.1. Protección de datos	49
11.2. Licencias de software	49
11.3. Propiedad intelectual	50
12. Manual de software	51
12.1. Despliegue	51
12.2. Uso del backend como librería	51
12.3. Exposición de una entidad y consumo del contrato	52
12.4. Generación del formulario en el frontend	52
12.5. Cómo escribir un plugin	53
12.6. Estructura de los repositorios	54
13. Otros capítulos y anexos	55
13.1. Ejemplos de capítulos opcionales	55
14. información academica	56
14.1. Ejemplo de sección - estilo de citas	56
14.1.1. Ejemplo de sub-sección	56
14.2. Ejemplo de sección - organización del documento	57
15. Anexos	59
15.1. Anexo A. Referencia de endpoints del sistema	59
15.2. Anexo B. Referencia de configuración	59
15.3. Anexo C. Acceso al código fuente	60

Listings

5.1. Diagrama textual de casos de uso	16
6.1. Flujo de datos extremo a extremo del framework	19
6.2. Integración de los plugins en el inicializador	21
6.3. Esqueleto del plugin de plantas (<code>plants/plugin.py</code>)	23
6.4. Esquema conceptual (textual) del núcleo	25
6.5. Secuencia de integración extremo a extremo	26
7.1. Docker Compose de la aplicación UniBack	28
7.2. Dockerfile de la aplicación UniBack	30
7.3. Envoltorio de respuesta normalizado (<code>responses.py</code>)	32
7.4. Envoltorio en éxito (arriba) y en error de validación (abajo)	32
7.5. Uso de la capa cliente genérica desde un componente Angular	34
7.6. SchemaService y DiscoveryService	34
7.7. Heurística de pistas de UI por columna (<code>utils/common.py</code>)	35
7.8. Formulario generado solo a partir del path de la entidad	38
7.9. Registro de entidad (<code>schema_registry.py</code>)	39
7.10. Documento del bundle de esquemas	39
12.1. Levantar el entorno completo	51
12.2. Inicialización y modelo propio del consumidor	51
12.3. Prueba rápida del contrato con curl	52
12.4. Formulario dinámico en una vista Angular	52
12.5. Estructura mínima de un plugin	53
12.6. Modelo de dominio de un plugin (<code>plants/models.py</code>)	53

Índice de Algoritmos

1.	Pseudocódigo con comentarios.	58
----	---------------------------------------	----

Índice de figuras

14.1. Logo vertical de la EII y la ULPGC [indicar crédito si no es una figura propia.
Internet o Google como buscador no es una fuente, habría que citar el origen
concreto] 57

Índice de cuadros

3.1. Grado de relación del TFT con los objetivos de desarrollo sostenible.	9
4.1. Comparación de enfoques de generación de interfaces.	13
5.1. Requisitos funcionales del framework.	15
5.2. Caso de uso CU-05: generar un formulario desde el path de entidad.	17
5.3. Caso de uso CU-07: crear o editar un registro vía formulario.	17
6.1. <i>Hooks</i> de ciclo de vida y extensiones inyectables de UnibackPlugin	21
6.2. Grupos de modelos del núcleo de <i>uniback</i>	24
7.1. Planificación por fases y estimación de esfuerzo.	28
7.2. Catálogo de códigos de error estándar del contrato.	33
7.3. Convención de endpoints por entidad.	33
7.4. Heurísticas de enriquecimiento del esquema por tipo de columna.	35
7.5. Mapeo de JSON Schema a tipos Formly.	37
7.6. Extensiones x-* reconocidas por el conversor.	37
7.7. Entradas del componente <ngt-dynamic-form>	38
7.8. Pruebas automatizadas por incremento.	41
8.1. Trabajo para exponer y editar una entidad nueva.	44
8.2. Grado de consecución de los objetivos específicos.	44
10.1. Estimación frente a esfuerzo real y estado por fase.	47
10.2. Presupuesto estimado del proyecto.	48
14.1. Ejemplo de tabla blablabla.	56
15.1. Endpoints transversales del contrato.	59
15.2. Secciones de configuración del inicializador.	60

Capítulo 1

Introducción

“Los buenos programadores saben qué escribir. Los grandes saben qué reutilizar.”

Eric S. Raymond [8]

1.1. Contexto

En la actualidad el software de gestión está sometido a cuatro presiones simultáneas: bajo coste, alta seguridad, alta eficiencia y un tiempo de desarrollo cada vez más reducido. El ritmo de evolución de las tecnologías y de los requisitos de negocio exige la capacidad de generar software de alta calidad en muy poco tiempo. Esto supone, en muchas ocasiones, una antítesis: para refinar y testear correctamente un producto es necesaria una inversión de tiempo que el calendario rara vez concede.

Para mitigar esta tensión la industria ha creado los Frameworks. Estos productos de software empaquetan las funcionalidades transversales necesarias para el desarrollo rápido (persistencia, seguridad, comunicación, validación) permitiendo asegurar y probar la base del código y obtener así productos firmes, de bajo coste y alta calidad. Sin embargo, a pesar de tratarse de software muy maduro, los frameworks comerciales de propósito general tienen límites: es imposible crear una solución universal que satisfaga las necesidades individuales de cada desarrollo, por lo que siempre es necesaria la introducción de arquitecturas y convenciones propias que, apoyándose en la herramienta, completen los requisitos concretos del proyecto.

Es en ese punto donde se sitúa el presente Trabajo de Fin de Grado: en el desarrollo de las herramientas y funcionalidades añadidas que completan un framework propio, ajustado a las arquitecturas y a la forma de trabajar del Instituto Tecnológico de Canarias (ITC) en sus proyectos de gestión de datos científicos.

1.2. Descripción del problema

El ITC, en colaboración con el Jardín Botánico Viera y Clavijo (JBVC), desarrolla aplicaciones web para la gestión de datos botánicos y de biodiversidad. Estos proyectos comparten una característica problemática: su modelo de datos no es estable, sino que *evoluciona* a medida que avanza la investigación, cambian los estándares y se incorporan nuevas fuentes y necesidades. Cada cambio en el modelo de datos propaga un coste en cascada que afecta al backend (modelos, validación, endpoints), a la capa de comunicación (contratos, tipos) y al frontend (formularios, tablas, vistas de detalle). Buena parte de ese código se escribe manualmente y se acopla a un dominio concreto, de modo que su reutilización en otro proyecto, o incluso su mantenimiento dentro del mismo, resulta caro y propenso a errores.

El problema que motiva este trabajo puede enunciarse así: *cómo reducir al mínimo el código que hay que escribir y mantener tanto para crear un proyecto nuevo, como para exponer, validar, comunicar y editar una entidad de datos cuando el modelo está en evolución constante, sin sacrificar la calidad ni acoplar la solución a un dominio particular.*

1.3. Solución propuesta

La solución que se aborda es un framework *full-stack* titulado UniWorks formado por dos piezas que se diseñan para encajar entre sí mediante un contrato uniforme:

- Un backend, ***uniback***, construido sobre FastAPI, SQLAlchemy2 y PostgreSQL, que aporta un núcleo de modelos reutilizables, un envoltorio de respuesta normalizado, fábricas de endpoints CRUD y la capacidad de derivar automáticamente un JSON Schema enriquecido a partir del modelo ORM.
- Una librería de componentes Angular, ***ngt-gui***, que consume ese contrato mediante una capa cliente genérica y, a partir del esquema publicado por el backend, genera formularios dinámicos con Angular Formly sin necesidad de escribir HTML específico de cada entidad.

La idea es que el modelo de datos sea la *única fuente de verdad*: un cambio en el ORM se propaga automáticamente al esquema, al contrato y a la interfaz, eliminando el código repetitivo y el riesgo de desincronización.

1.4. Estructura del documento

El resto del documento se organiza como sigue. El capítulo **Estado actual y objetivos iniciales** describe los antecedentes del proyecto, su punto de partida y los objetivos generales y específicos. El capítulo de **Aportaciones** trata la repercusión del trabajo, las competencias cubiertas y su alineamiento con los “Objetivos de Desarrollo Sostenible”. El capítulo de

Desarrollo constituye el núcleo técnico de la memoria: presenta la metodología, la planificación, el análisis de requisitos, la arquitectura y el detalle de cada uno de los incrementos. A continuación, el capítulo de **Resultados** valora lo conseguido, y el de **Conclusiones y trabajo futuro** recoge las lecciones aprendidas, las líneas de desarrollo futuras y el uso que se ha hecho de la Inteligencia Artificial. El documento finaliza con la bibliografía y los anexos.

Capítulo 2

Estado actual y objetivos iniciales

2.1. Motivación y antecedentes

Este proyecto surge como una expansión del desarrollo ejercido en mis prácticas de empresa en el ITC. La librería “**NextGenDem-GUI-LIB**” fue diseñada originalmente para facilitar y acelerar el desarrollo Frontend sobre un Backend específico, el del proyecto NextGenDem, dedicado a la gestión de datos bioinformáticos.

Este planteamiento, limitaba su aplicabilidad a proyectos con arquitecturas prácticamente idénticas: cualquier modificación en el backend obligaba a reescribir una cantidad notable de código en el frontend, porque los componentes conocían de forma explícita la forma de los datos y los endpoints concretos del proyecto original. Reconociendo esta limitación, y tras consultarlo con mi tutor de empresa del ITC, se planteó la posibilidad de ampliar el alcance del proyecto para construir un Framework completo que incluyese tanto backend como frontend, de manera que pudiera reutilizarse en una variedad mucho más amplia de proyectos y arquitecturas. Además, la llegada de un nuevo proyecto con el Jardín Botánico Viera y Clavijo manifestó la utilidad de un proyecto de este tipo.

Por otra parte, el desarrollo ya estaba avanzado y, debido a las limitaciones temporales de las prácticas de empresa, tuve que redirigir el trabajo hacia un punto intermedio, ya que priorizé entregar la librería NextGenDem-GUI-LIB funcional. Así, el proyecto original se orientó a crear una librería frontend de componentes reutilizables. Sobre la base de ese desarrollo se planteó el presente Trabajo de Fin de Grado, con el objetivo de consolidar, documentar y presentar el framework de forma completa, backend y frontend, y demostrar su utilidad ante un modelo de datos en evolución.

2.1.1. El ecosistema previo: NextGenDem, uniback y ngt-gui

El punto de partida lo forman dos artefactos. Por un lado, un backend heredado en Python basado en el backend de NextGenDem con una base de modelos de dominio (ob-

jetos funcionales, autenticación y autorización, anotaciones, tareas) pero sin una capa de comunicación uniforme ni mecanismos de exposición automática. Por otro, la librería Angular NextGenDem-GUI-LIB, con un catálogo amplio de componentes de presentación (tablas, menús, diálogos, layout) pero acoplada a un servicio de backend monolítico con más de doscientos métodos escritos a mano, uno por cada operación concreta del proyecto original.

2.1.2. La colaboración con el Jardín Botánico Viera y Clavijo

El JBVC es la institución de conservación e investigación botánica principal en Canarias. Sus necesidades — catalogación de especies, gestión de colecciones, anotaciones de campo, jerarquías taxonómicas — son un caso de uso ideal de un modelo de datos en evolución ya que el esquema crece y se ajusta a medida que avanza la digitalización de sus fondos y los propios estándares en los que se definen los datos. El framework no es la solución final para el Jardín, sino la herramienta con la que dicha solución se construye; por ello la interacción con esta institución sirvió para calibrar el grado de abstracción y genericidad que el framework debía ofrecer.

2.2. Estado del desarrollo

El desarrollo del framework se encontraba, al inicio del TFG, en una fase intermedia y *asimétrica*. El frontend tenía una buena parte de sus funcionalidades implementadas, incluyendo componentes básicos y algunas funcionalidades avanzadas, pero seguía acoplado al backend original. El backend, que no se había abstraído, funcionaba como un backend tradicional: se me entregó una base razonablemente completa de modelos ORM, pero sin una capa de API homogénea, porque el ITC había priorizado disponer de un backend mínimamente funcional para arrancar su último proyecto con recursos limitados.

En resumen, el framework no estaba terminado y, a excepción del frontend, no disponía de una base de código suficiente para considerarse utilizable de forma genérica. Es aquí donde comienza mi desarrollo en el presente Trabajo de Fin de Grado, con el objetivo de completar, optimizar y cerrar tanto el backend como el frontend, y de unirlos mediante un contrato que los desacople del dominio.

2.3. Planificación previa al desarrollo

Antes de iniciar el desarrollo se realizó una planificación preliminar con el objetivo de establecer una hoja de ruta clara y estructurada que permitiera, posteriormente, una planificación completa del proyecto.

2.3.1. Análisis previo

2.3.1.1. Frontend

Dado que el desarrollo del frontend lo realicé yo durante mis prácticas de empresa, soy plenamente conocedor de su alcance, su arquitectura interna y de las tareas necesarias para su finalización, lo que redujo la incertidumbre de estimación de esta parte.

2.3.1.2. Backend

El backend, al ser un desarrollo que no realicé yo, presentaba mayor incertidumbre: no tenía un conocimiento profundo de su alcance ni de las tareas pendientes. Por ello contacté con mi tutor de empresa en el ITC para obtener una visión clara del estado real, de la deuda técnica y de los objetivos prioritarios.

2.3.2. Acuerdo de planificación evolutiva

Tras el contacto con la empresa se estableció una metodología iterativa incremental: a medida que avanzase el desarrollo y surgiesen nuevas necesidades o desafíos, se irían ajustando y refinando los objetivos y tareas. Esta planificación dinámica es coherente con la naturaleza del problema — un modelo de datos que evoluciona — y permite mantener el proyecto alineado con las necesidades reales en cada momento.

2.3.3. Tareas iniciales

Se definieron las tareas iniciales a partir del análisis previo y del *feedback* del ITC. Debido a la naturaleza evolutiva de la planificación, estas tareas llevaron a divergir ligeramente, en ocasiones, de la planificación inicial propuesta en el TFT01 y a elaborar una planificación más ajustada a las necesidades reales, que se detalla en el capítulo de Desarrollo.

2.4. Objetivos

Tras contactar con mi tutor de prácticas, mi tutor académico y el Jardín Botánico Viera y Clavijo, se establecieron los objetivos generales siguientes:

1. Diseñar e implementar un framework Backend/Frontend reutilizable.
2. Proporcionar una arquitectura tecnológica extensible y desacoplada del dominio.
3. Adaptar dinámicamente el desarrollo para asegurar su utilidad ante necesidades emergentes.

Asimismo se definieron los siguientes objetivos específicos:

1. **Crear una API modular.** Establecer un envoltorio de respuesta uniforme, un catálogo de errores estándar y fábricas de endpoints CRUD que eliminen el código repetitivo por entidad.
2. **Diseñar y modelar los esquemas de datos.** Consolidar el núcleo de modelos ORM y dotarlos de versionado e histórico.
3. **Implementar un sistema de comunicación estandarizado** entre backend y frontend, incluyendo descubrimiento de entidades y una capa cliente genérica tipada.
4. **Diseñar, refactorizar y mejorar el generador automático de formularios** basado en los componentes Formly y Grid, guiado por el esquema publicado por el backend.
5. **Refactorizar o reimplementar y mejorar los mecanismos para la generación automática de esquemas** a partir del modelo ORM, de forma que el modelo de datos sea la única fuente de verdad.

Capítulo 3

Aportaciones del trabajo

3.1. Principales aportaciones

La principal aportación de este Trabajo de Fin de Grado es un framework *full-stack* reutilizable que reduce drásticamente el código que hay que escribir y mantener para exponer, validar, comunicar y editar una entidad de datos cuando el modelo está en evolución. Concretamente, el trabajo aporta:

- Un **contrato de comunicación uniforme** (envoltorio de respuesta, catálogo de errores y descubrimiento) que desacopla el frontend del dominio del backend.
- Una **cadena de generación automática** que convierte el modelo ORM en esquema enriquecido y este, sin intervención manual, en formularios renderizables.
- Una **capa cliente genérica tipada** en Angular que convive con el código heredado sin romperlo, facilitando una migración progresiva.
- Un **entorno de despliegue reproducible** multiplataforma con Docker, validado en sistemas operativos y arquitecturas distintas.

La repercusión esperada es doble. En el plano técnico, el ITC dispone de una base que acorta el tiempo de arranque y el coste de mantenimiento de sus futuros proyectos de gestión de datos. En el plano socio-económico, el primer beneficiario es el JBVC, cuya labor de conservación e investigación de la flora canaria se apoyará en aplicaciones construidas más rápido, con menos errores y con mayor calidad y adaptabilidad.

3.2. Competencias específicas

El trabajo cubre, de forma directa, varias competencias expuestas en el plan de estudios del Grado en Ingeniería Informática. En concreto, se abordan las siguientes competencias

comunes a la rama de la informática y de la tecnología de la información: **CI1, CI2, CI3, CI5, CI6, CI7, CI9, CI12, CI13, CI14, CI16, CI17, TI1, TI3, TI5, TI6 y TI7.**

En la **ingeniería del software** se ejercita el análisis de requisitos, el diseño arquitectónico en capas, la aplicación de una metodología iterativa incremental y la verificación mediante pruebas automatizadas en backend y frontend. En **sistemas de información** se diseña y modela un esquema de datos relacional con versionado e histórico, y se gestiona su evolución con migraciones. En **ingeniería de computadores y tecnología** se aborda el despliegue reproducible mediante contenedores y una arquitectura preparada para microservicios. Transversalmente se cubren competencias de comunicación técnica (documentación del contrato y de los incrementos) y de trabajo en colaboración con una empresa y un cliente reales.

3.3. Alineamiento con los objetivos de desarrollo sostenible

Cuadro 3.1: Grado de relación del TFT con los objetivos de desarrollo sostenible.

ODS	Grado de relación			
	0 No procede	1 Bajo	2 Medio	3 Alto
1 Fin de la Pobreza	X			
2 Hambre cero	X			
3 Salud y Bienestar	X			
4 Educación de calidad	X			
5 Igualdad de género	X			
6 Agua limpia y saneamiento	X			
7 Energía Asequible y no contaminante	X			
8 Trabajo decente y crecimiento económico			X	
9 Industria, Innovación e Infraestructuras				X
10 Reducción de las desigualdades	X			
11 Ciudades y comunidades sostenibles	X			
12 Producción y consumo sostenibles		X		
13 Acción por el clima	X			
14 Vida submarina	X			
15 Vida de ecosistemas terrestres			X	
16 Paz, justicia e instituciones sólidas	X			
17 Alianzas para lograr objetivos	X			

De acuerdo con la tabla 3.1, el TFT presenta un grado de relación **alto** con el ODS 9 (Industria, innovación e infraestructuras), ya que aporta infraestructura software reutilizable que fomenta la innovación y la eficiencia en el desarrollo. Presenta un grado **medio** con el ODS 4 (Educación de calidad), por su valor formativo y por documentar prácticas de ingeniería reutilizables, y con el ODS 15 (Vida de ecosistemas terrestres), puesto que las

aplicaciones construidas con el framework apoyan directamente la conservación y el estudio de la flora endémica de Canarias por parte del JBVC. El resto de objetivos no presentan una relación directa significativa con el trabajo desarrollado.

Capítulo 4

Marco tecnológico y estado del arte

Este capítulo sitúa el trabajo en su contexto técnico: las tecnologías sobre las que se construye el framework y los enfoques existentes para el problema de generar interfaces a partir de un modelo de datos.

4.1. Tecnologías del backend

4.1.1. FastAPI y Pydantic

FastAPI [7] es un *framework* web asíncrono para Python que genera documentación OpenAPI automáticamente y valida las entradas y salidas mediante Pydantic. La elección es relevante para este trabajo porque Pydantic puede emitir JSON Schema [5] a partir de sus modelos, lo que convierte a FastAPI en un punto natural para publicar el esquema de cada entidad. La inyección de dependencias de FastAPI se emplea para gestionar las sesiones de base de datos y la autenticación de forma transversal.

4.1.2. SQLAlchemy 2.x y Alembic

SQLAlchemy [1] es el ORM de referencia en Python. Su versión 2.x introduce la API tipada con `Mapped[...]` y `mapped_column`, que el framework aprovecha para inferir tipos al construir el esquema. Alembic gestiona las migraciones de esquema, imprescindible cuando el modelo de datos evoluciona: cada cambio se materializa en una *revision* versionada y reproducible.

4.1.3. SQLAlchemy-Continuum

SQLAlchemy-Continuum añade versionado e histórico automático a los modelos ORM. Su particularidad — debe inicializarse antes de definir los modelos — es una restricción de

diseño que condicionó la arquitectura del inicializador. El versionado nativo delega el histórico en el propio motor cuando es posible, reduciendo el coste de mantener tablas espejo.

4.1.4. Infraestructura: PostgreSQL, Docker, Redis y Celery

PostgreSQL es la base de datos relacional elegida por su robustez y su soporte de tipos avanzados (JSONB, *full-text search*). Docker y `docker compose` aportan despliegue reproducible multiplataforma (RNF-02). Redis y Celery habilitan el procesamiento asíncrono de tareas (`SystemProcess`), inicializados de forma opcional por el framework.

4.2. Tecnologías del frontend

4.2.1. Angular

Angular [4] es un *framework* de aplicaciones web basado en TypeScript, con inyección de dependencias y un sistema de módulos que encaja con la distribución del proyecto como librería publicable (`ngt-gui`). Su tipado fuerte permite que la capa cliente del contrato sea genérica y tipada.

4.2.2. Formly

Angular Formly [3] construye formularios de manera declarativa a partir de una configuración (`FormlyFieldConfig[]`) en lugar de HTML campo a campo. Soporta tipos personalizados, validadores y expresiones reactivas, lo que lo hace idóneo como destino de una traducción automática desde el esquema. El proyecto lo combina con *ng-zorro* y un conjunto de tipos personalizados.

4.3. Estado del arte: generación de interfaces dirigida por esquema

La idea de derivar interfaces de un esquema no es nueva. En el ecosistema web existen `react-jsonschema-form` y `JSON Forms`, que renderizan formularios a partir de JSON Schema. Estas soluciones, sin embargo, se centran en el lado cliente y asumen que el esquema se proporciona ya elaborado. En el lado servidor, herramientas como FastAPI exponen OpenAPI, pero ese contrato describe *operaciones*, no incluye pistas de presentación, y no resuelve el acoplamiento del cliente al dominio.

La tabla 4.1 compara los enfoques. La aportación diferencial de este trabajo no es inventar un renderizador de esquemas, sino **cerrar la cadena completa**: derivar el esquema *desde*

el *ORM*, enriquecerlo con pistas de presentación en el propio esquema, transportarlo por un contrato uniforme con versionado, y consumirlo con una capa genérica que mantiene la compatibilidad con el código heredado.

Cuadro 4.1: Comparación de enfoques de generación de interfaces.

Enfoque	Desde ORM	Hints UI	Contrato	Versionado
react-jsonschema-form	No	Parcial	No	No
JSON Forms	No	Sí (uischema)	No	No
OpenAPI/Swagger UI	No	No	Operaciones	No
Admin autogenerados	Sí	Limitado	Acoplado	No
Este trabajo	Sí	Sí (x-*)	Uniforme	ETag

Frente a los *admin* autogenerados (al estilo del de Django), que sí parten del ORM pero acoplan la interfaz al servidor y son difíciles de reutilizar fuera de su *stack*, la solución propuesta mantiene frontend y backend desacoplados mediante un contrato explícito y portable, que es precisamente lo que habilita la reutilización en proyectos con modelos de datos en evolución.

Capítulo 5

Análisis de requisitos

Este capítulo recoge el análisis de requisitos del framework. Dado que el producto no es una aplicación final sino una *herramienta de construcción*, los requisitos se expresan en términos de las capacidades que el framework debe ofrecer a los desarrolladores que lo usen y, de forma indirecta, a los usuarios finales de las aplicaciones construidas con él.

5.1. Identificación de *stakeholders*

Se identificaron los siguientes Stakeholders:

- **El ITC** como propietario del framework y principal consumidor: necesita una base reutilizable que reduzca el coste de arrancar y mantener nuevos proyectos.
- **El equipo de desarrollo** que construirá aplicaciones sobre el framework: necesita una API predecible, tipada y bien documentada.
- **El JBVC** como cliente del primer proyecto real construido con el framework: aporta el caso de uso que valida la genericidad.
- **El usuario final** de las aplicaciones: necesita formularios y vistas correctas, validadas y coherentes, aunque nunca interactúa con el framework directamente.

5.2. Requisitos funcionales

La tabla 5.1 resume los requisitos funcionales priorizados según el esquema MoSCoW (*Must, Should, Could*).

Cuadro 5.1: Requisitos funcionales del framework.

ID	Descripción	Prioridad
RF-01	Toda respuesta de la API debe seguir un envoltorio uniforme con contenido, recuento y lista de incidencias.	Must
RF-02	El framework debe ofrecer fábricas que generen los endpoints CRUD de una entidad sin escribir un router a mano.	Must
RF-03	Cada entidad debe publicar su JSON Schema en un endpoint dedicado.	Must
RF-04	El backend debe exponer un endpoint de descubrimiento que liste las entidades disponibles y sus capacidades.	Must
RF-05	El frontend debe disponer de una capa cliente genérica tipada que consuma cualquier entidad a partir de su <i>path</i> .	Must
RF-06	El frontend debe generar un formulario completo a partir del esquema de una entidad, sin HTML específico.	Must
RF-07	El esquema debe poder enriquecerse con pistas de presentación (<i>widget</i> , opciones, visibilidad condicional).	Should
RF-08	El backend debe ofrecer un <i>bundle</i> con los esquemas de todas las entidades, versionado para <i>caching</i> .	Should
RF-09	El sistema debe soportar autenticación (Firebase y un modo local de desarrollo) y autorización basada en roles y ACL.	Should
RF-10	Las entidades núcleo deben mantener histórico de cambios (versionado).	Could

5.3. Requisitos no funcionales

RNF-01 (Reutilización). El framework no debe acoplar ninguna lógica a un dominio concreto; el dominio se introduce mediante modelos del consumidor, articulados a través del sistema de plugins descrito en la sección 6.3.

RNF-02 (Portabilidad). El sistema completo debe desplegarse de forma reproducible en Windows, macOS y Linux mediante Docker.

RNF-03 (Mantenibilidad). El modelo de datos debe ser la única fuente de verdad; el contrato y la interfaz deben derivarse de él.

RNF-04 (Fiabilidad). Las piezas críticas deben estar cubiertas por pruebas automatizadas en backend y frontend.

RNF-05 (Eficiencia). El contrato debe permitir *caching* agresivo de los esquemas mediante validadores condicionales (ETag).

RNF-06 (Compatibilidad). La nueva capa cliente debe convivir con el servicio de backend heredado sin romperlo.

RNF-07 (Seguridad). Las credenciales y la verificación de tokens no deben quedar expuestas; el modo de desarrollo local no debe estar activo en producción.

5.4. Casos de uso

Aunque el framework es una herramienta para desarrolladores, su valor se entiende mejor a través de los casos de uso que habilita. Se distinguen dos actores: el *desarrollador consumidor*, que construye una aplicación usando el framework, y el *usuario final*, que interactúa con la aplicación resultante. El Algoritmo 5.1 resume las relaciones.

```
Desarrollador consumidor
|-- CU-01 Declarar una entidad (modelo ORM)
|-- CU-02 Exponerla via fabrica CRUD
|-- CU-03 Obtener su esquema enriquecido
|-- CU-04 Descubrir entidades disponibles
|-- CU-05 Generar formulario desde el path
'-- CU-06 Desplegar el sistema (docker compose)

Usuario final (de la app construida)
|-- CU-07 Crear/editar un registro vía formulario
|-- CU-08 Listar/filtrar registros
'-- CU-09 Autenticarse (Firebase / local dev)
```

Algoritmo 5.1: Diagrama textual de casos de uso

A continuación se detallan los casos de uso más representativos.

Cuadro 5.2: Caso de uso CU-05: generar un formulario desde el path de entidad.

Identificador	CU-05
Actor	Desarrollador consumidor
Precondición	La entidad está expuesta vía fábrica CRUD y publica su <code>schema.json</code> .
Flujo principal	1) El desarrollador coloca <code><ngt-dynamic-form entityPath=/x></code> . 2) El componente pide el esquema a <code>SchemaService</code> . 3) <code>schemaToFormly</code> traduce el esquema. 4) Se renderiza el formulario con validadores.
Postcondición	El usuario final dispone de un formulario funcional sin que se haya escrito HTML específico.
Flujo alternativo	Si el esquema trae <code>issues</code> de tipo <code>ERROR</code> , <code>SchemaService</code> propaga el error y el componente muestra las incidencias.

Cuadro 5.3: Caso de uso CU-07: crear o editar un registro vía formulario.

Identificador	CU-07
Actor	Usuario final
Precondición	Existe un formulario generado para la entidad.
Flujo principal	1) El usuario rellena el formulario. 2) Los validadores derivados del esquema comprueban los datos. 3) Al enviar, se invoca <code>EntityClient.create()/update()</code> . 4) El backend valida con Pydantic y responde con el envoltorio.
Postcondición	El registro queda persistido y versionado; las incidencias se muestran bajo el formulario.

5.5. Restricciones

El proyecto está condicionado por el stack tecnológico ya existente (Python/FastAPI/SQ-LAlchemy en backend y Angular en frontend), por el calendario académico del TFG y por la necesidad de no introducir regresiones en el código heredado que la empresa ya utiliza en producción.

Capítulo 6

Diseño y arquitectura

6.1. Visión general

La arquitectura del framework se organiza en dos artefactos desplegables — el backend *uniback* y el frontend *ngt-gui* — unidos por un **contrato** explícito que actúa como única superficie de acoplamiento. El principio rector del diseño es que el *modelo de datos* (los modelos ORM de SQLAlchemy) es la única fuente de verdad: de él se derivan, de forma automática, el esquema, el contrato de la API y los formularios de la interfaz. El Algoritmo 6.1 representa el flujo de datos completo.

```
Modelo SQLAlchemy (fuente de verdad)
    | make_simple_rest_crud / make_crudie_rest_crud
    v
sqlalchemy_to_pydantic --> Pydantic + hints x-* por columna
    |
    v
get_enriched_json_schema --> JSON Schema enriquecido
    |
    +--> GET /<entity>/schema.json (ETag)
    +--> GET /api/sys/schemas (bundle versionado)
    +--> GET /api/sys/entities (descubrimiento)
    |
    v [ ResponseEnvelope: content / count / issues ]
----- contrato HTTP -----
    |
    v
ngt-gui: EntityClient / SchemaService / DiscoveryService
    |
    v
schemaToFormly(schema) --> FormlyFieldConfig[]
    |
    v
```

```
<ngt-dynamic-form> --> formulario renderizado
```

Algoritmo 6.1: Flujo de datos extremo a extremo del framework

6.2. Arquitectura del backend (*uniback*)

El backend sigue una arquitectura en capas dentro de un paquete Python instalable, construido sobre FastAPI [7] y SQLAlchemy 2.x [1]. Las capas principales son:

Persistencia. Modelos ORM SQLAlchemy 2.x organizados por dominio (*core*, *sysadmin*, *annotations*, *hierarchies*, *species*, *files*, *geographics*, *screens*). Un *BaseMixin* aporta funcionalidad común (UUID, marcas de tiempo, atributos). El prefijo de tablas es configurable (*ub_* por defecto) y SQLAlchemy-Continuum proporciona el histórico de cambios.

Servicios. Lógica de dominio (autenticación, colecciones, jerarquías, importación, anotaciones) desacoplada de la capa HTTP.

API. FastAPI: *routers*, esquemas Pydantic, manejadores de error globales, fábricas CRUD y registro de entidades.

Inicializador. Un único punto de entrada *initialize(config)* que normaliza la configuración (Pydantic Settings), prepara la base ORM, el gestor de sesiones y la aplicación FastAPI, e inicializa de forma opcional Redis, Celery y Firebase.

El despliegue se realiza con Docker mediante *docker compose*, con servicios para PostgreSQL, Redis, el *worker* Celery y el servidor web, lo que garantiza reproducibilidad multiplataforma (RNF-02). Existe además un *api_gateway* con configuración Nginx para escenarios de microservicios y un mecanismo de descubrimiento de servicios externos a través de Traefik.

6.3. Sistema de plugins y extensibilidad

El framework no debe acoplar ninguna lógica a un dominio concreto (RNF-01); el dominio se introduce *desde fuera* del núcleo mediante un sistema de plugins. Un plugin es un módulo Python autónomo que se engancha al ciclo de vida del inicializador para aportar modelos, *endpoints*, datos semilla y extensiones de comportamiento, sin que el núcleo conozca su existencia. Es la pieza que materializa la idea, heredada de la transición desde NextGenDem, de que el caso de uso del JBVC sea una capa apilada sobre *uniback* y no una modificación de él.

6.3.1. El contrato del plugin

Todo plugin hereda de la clase base *UnibackPlugin* y sobrescribe solo los *hooks* que necesita; el resto devuelven listas vacías o no hacen nada, lo que preserva la retrocompatibi-

lidad. Los *hooks* se dividen en eventos de ciclo de vida y *getters* de extensiones inyectables, resumidos en la tabla 6.1.

Cuadro 6.1: *Hooks* de ciclo de vida y extensiones inyectables de `UnibackPlugin`.

Hook	Tipo	Propósito
<code>on_init(settings)</code>	ciclo de vida	Configuración temprana y servicios de terceros.
<code>get_model_modules()</code>	ciclo de vida	Módulos con modelos ORM a registrar antes de <code>configure_mappers()</code> .
<code>get_routers()</code>	ciclo de vida	<i>Routers</i> FastAPI a montar en la aplicación.
<code>on_seed(db)</code>	ciclo de vida	Inyectar filas por defecto durante el <i>auto-seed</i> .
<code>on_app_ready(app)</code>	ciclo de vida	Middlewares o montajes tras crear la app.
<code>get_source_adapters()</code>	extensión	Adaptadores que abren <i>streams</i> desde orígenes externos.
<code>get_importers(), get_exporters()</code>	extensión	Parsers y serializadores para el insertador genérico.
<code>get_field_widgets()</code>	extensión	Pistas de UI que enriquecen el JSON Schema generado.
<code>get_fk_resolvers()</code>	extensión	Resolvers de claves foráneas (<code>{by, value}</code>) en la importación.

6.3.2. Descubrimiento e integración en el inicializador

El `PluginManager` (un *singleton* global) descubre los plugins escaneando un directorio configurable (variable `UNIBACK_PLUGINS_PATH`, por defecto `src/plugins`): por cada paquete importa su submódulo `plugin` y registra las subclases de `UnibackPlugin` que encuentra. A partir de ahí, `initialize(config)` emite los eventos del ciclo de vida en un orden preciso, intercalándolos con la construcción de la base ORM y de la aplicación FastAPI. El Algoritmo 6.2 resume esa secuencia.

```

initialize(config)
|
v
discover_plugins(UNIBACK_PLUGINS_PATH | src/plugins)
| importa <plugin>.plugin y detecta subclases de UnibackPlugin
v
emit_on_init(settings) # hook on_init de cada plugin
register_core_defaults() # importers/resolvers del nucleo
populate_registries() # vuelca los get_* en los registros globales
|

```

```

v
create_orm_base() + import modelos del nucleo y de los plugins #
    get_model_modules
|
v
configure_mappers() # polimorfismo + versionado (Continuum)
|
v
create_tables() (si procede)
|
v
auto_seed: emit_on_seed(db) # hook on_seed dentro del session_scope
|
v
create_app(...) # monta los get_routers() de los plugins
|
v
emit_on_app_ready(app) # hook on_app_ready

```

Algoritmo 6.2: Integración de los plugins en el inicializador

El orden es deliberado: los modelos del plugin (`get_model_modules`) se importan *antes* de `configure_mappers()`, de modo que sus entidades queden registradas en la misma base ORM que las del núcleo y participen del polimorfismo y del versionado; `on_seed` corre dentro del *scope* de sesión del inicializador, junto a la siembra del núcleo; y `on_app_ready` se invoca con la aplicación ya construida, momento en el que el plugin puede montar middlewares o ficheros estáticos.

6.3.3. Extensiones inyectables y registros

Más allá de los *hooks* de ciclo de vida, un plugin puede aportar implementaciones de cinco contratos (`SourceAdapter`, `Importer`, `Exporter`, `FieldWidget` y `FKResolver`) que el `PluginManager` vuelca en sendos registros globales *thread-safe*. La política de selección es “el último registrado para un mismo `name` gana”, de modo que un plugin puede sobrescribir un comportamiento por defecto del núcleo simplemente registrando otra implementación con el mismo nombre. El núcleo inscribe primero sus *defaults* (`JsonImporter`, `NdjsonImporter` y dos resolvers de claves foráneas), de manera que la importación genérica funciona *out of the box* sin ningún plugin presente. Estos contratos son los puntos de extensión que permiten, por ejemplo, introducir un nuevo formato de importación (CSV, XLSX) o un tipo de *widget* de UI (un mapa, un editor de código) sin tocar el conversor de esquema ni las fábricas CRUD.

6.3.4. Ejemplo: el plugin de plantas

El plugin `PlantsApp` es una aplicación de demostración que añade, como capa externa, un catálogo de plantas geolocalizadas — un dominio cercano al del JBVC. Sin modificar una

sola línea del núcleo, el plugin (i) registra el modelo `Plant` (subclase de `FunctionalObject`); (ii) expone su CRUD REST completo con `make_simple_rest_crud`, incluyendo `/schema.json` y `/bulk`; (iii) aporta un `FieldWidget` que enriquece las columnas de coordenadas con pistas `x-*` para el mapa; y (iv) siembra datos de ejemplo y una entrada en el menú lateral. El Algoritmo 6.3 muestra su esqueleto.

```
class PlantsPlugin(UnibackPlugin):
    name = "PlantsApp"
    version = "1.0.0"

    # 1) Modelos: registrados antes de configure_mappers()
    def get_model_modules(self):
        return ["plants.models"]

    # 2) API: CRUD REST completo + /schema.json + /bulk
    def get_routers(self):
        from uniback.api.crud_factory import make_simple_rest_crud
        from plants.models import Plant
        return [make_simple_rest_crud(Plant, "plants", tags=["Plants"])]

    # 3) Pista de UI inyectable para las coordenadas del mapa
    def get_field_widgets(self):
        return [MapCoordinatesWidget()]

    # 4) Semilla: ObjectType, datos demo y menu lateral
    def on_seed(self, db):
        ...
```

Algoritmo 6.3: Esqueleto del plugin de plantas (`plants/plugin.py`)

Este ejemplo cierra el argumento de reutilización (RNF-01): exponer un dominio nuevo y editable desde el frontend se reduce a escribir un plugin que declara su modelo y lo expone con una fábrica CRUD; el contrato, el esquema enriquecido y el formulario dinámico se derivan automáticamente, tal como describe el resto del capítulo.

6.4. Arquitectura del frontend (*ngt-gui*)

El frontend es una librería Angular [4] publicable (`projects/ngt-gui`) acompañada de una aplicación de demostración, que emplea Angular Formly [3] para construir los formularios a partir del JSON Schema [5]. La librería se estructura en `core` (lógica sin dependencia de UI: conversor de esquema, tipos del contrato), `services` (clientes HTTP, caché de esquemas, descubrimiento), `components` (catálogo de presentación, entre ellos `dynamic-form` y `formly-components`), `models`, `interfaces` y `tokens` de inyección.

6.4.1. Catálogo de modelos del núcleo

El backend agrupa los modelos por dominio y les asigna un código de tipo de objeto (`ObjectType`) que habilita el tratamiento polimórfico. La tabla 6.2 resume los grupos principales y su finalidad.

Cuadro 6.2: Grupos de modelos del núcleo de *uniback*.

Módulo	Contenido
<code>core</code>	<code>ObjectType</code> , <code>FunctionalObject</code> : base común con UUID, propiedad, marcas de tiempo y atributos.
<code>sysadmin</code>	Autenticación/autorización: <code>Authorizable</code> , <code>Identity</code> , <code>Role</code> , <code>Organization</code> , <code>Group</code> , <code>ACL</code> .
<code>annotations</code>	Plantillas, campos y textos de anotación sobre objetos funcionales.
<code>hierarchies</code>	Jerarquías (p. ej. taxonómicas) y colecciones.
<code>species</code>	Modelos de especies (incl. soporte Darwin Core).
<code>files</code>	Sistema de ficheros: <code>FileSystemObject</code> , <code>Folder</code> , <code>File</code> .
<code>screens</code>	Modelos de GUI: <i>app flavor</i> , menús, pantallas.
<code>geographics</code>	Datos geográficos asociados a las entidades.

6.4.2. Catálogo de la librería de componentes

La librería `ngt-gui` se organiza en módulos funcionales: `core` (conversor de esquema y tipos del contrato, sin dependencia de UI), `services` (`EntityClient`, `SchemaService`, `DiscoveryService`, interceptor de error), `components` (`dynamic-form`, `formly-components` con ~20 tipos Formly personalizados sobre *ng-zorro*, además de *layout*, *data*, *display*, *admin* y *auth*), `models`, `interfaces`, `pipes` y `tokens` de inyección. Esta separación garantiza que la lógica de generación (`core`) sea testeable de forma aislada (RNF-04).

6.5. Persistencia, sesiones y configuración

La capa de persistencia se apoya en una base ORM declarativa creada por `create_orm_base()` y en un gestor de sesiones (`SessionManager` / `create_session_factory`) que expone la sesión activa mediante un contexto (`current_session_ctx`). Esto permite que los modelos — por ejemplo, para resolver el propietario actual de un `FunctionalObject` — accedan a la sesión sin recibirla por parámetro, manteniendo el código de dominio limpio.

La configuración se modela con Pydantic Settings (`Settings`, `AuthSettings`, `WorkerSettings`), de modo que un diccionario o variables de entorno se validan y normalizan en un único punto. El inicializador acepta indistintamente un diccionario o una instancia `Settings`, lo que facilita tanto el uso programático como el despliegue en contenedores con variables `UNIBACK_*`.

6.6. Modelo polimórfico de objetos funcionales

El corazón del modelo de datos es la jerarquía de `FunctionalObject`. Cada entidad de dominio hereda de ella y queda asociada a un `ObjectType` con un código numérico (p. ej. `dataset=0`, `collection=106`, `screen=32`). Este diseño habilita el tratamiento homogéneo de entidades heterogéneas: anotaciones, ACL y colecciones pueden referirse a “cualquier objeto funcional” sin conocer su clase concreta. El Algoritmo 6.4 resume la relación de forma textual.

```

ObjectType (id, uuid, name)
  ^ 1
  | N
FunctionalObject (id, uuid, owner, created/updated, attrs)
  ^ ^
  | hereda | referido por
[Collection, Screen, ...] Annotation* / ACL*

Authorizable (name, uuid) <- tabla padre
  ^ ^

Identity Role <- herencia polimórfica

```

Algoritmo 6.4: Esquema conceptual (textual) del núcleo

6.7. Modelo de autorización

La autorización se basa en *Authorizables* (identidades y roles, con herencia polimórfica), organizaciones y grupos, y un modelo de permisos formado por `ACL`, `ACLExpression` y `ACLDetail` junto con `PermissionType` y `ObjectTypePermissionType`. Este diseño permite expresar permisos por tipo de objeto y por instancia, soportando el principio de mínimo privilegio. Existen roles e identidades de sistema con UUID fijos (p. ej. *sys-admin*, *guest*) para arranque y pruebas. La autenticación admite Firebase (verificación de *token*) y un modo local de desarrollo, claramente separado para no exponerse en producción (RNF-07).

6.8. El contrato como frontera de desacoplamiento

La decisión de diseño más relevante es interponer un **envoltorio universal** (*ResponseEnvelope*) entre ambos lados. Todo el tráfico, de éxito o de error, viaja con la misma forma — `content`, `count` e `issues` — y un catálogo de códigos de error estándar. Gracias a ello, el frontend nunca necesita conocer la forma concreta de una entidad para manejar respuestas y errores: solo el conversor de esquema, en un único punto, traduce datos a interfaz. Esta frontera es lo que hace posible cumplir los requisitos de reutilización (RNF-01) y mantenibilidad (RNF-03).

6.9. Escenario de integración extremo a extremo

El contrato completo se ejercita componiendo las piezas reutilizables de `ngt-gui`; en particular, el componente `<ngt-dynamic-form>` resuelve internamente el descubrimiento, la obtención y cacheo del esquema, la conversión a Formly y el envío mediante `EntityClient`. El Algoritmo 6.5 describe la secuencia de interacción resultante, que se verifica con las pruebas automatizadas del conversor y del bundle.

```
Vista DiscoveryService SchemaService EntityClient Backend
| loadEntities() | | | |
|----->|---- GET /api/sys/entities ----->|
| EntityDescriptor[] <-----|
| selecciona "roles" | | | |
|---- get('/roles') ----->| GET /roles/schema.json -->|
| JSON Schema (cache) <-----|<-- ETag / 304 -----|
| schemaToFormly(schema) -> fields | | |
| usuario rellena y envia | | |
|---- create(model) ----->| POST /roles/ ->|
| ResponseEnvelope (content/issues) <-----|<-----|
```

Algoritmo 6.5: Secuencia de integración extremo a extremo

Este escenario demuestra que las cuatro piezas — descubrimiento, esquema cacheado con ETag, conversión a Formly y cliente tipado — encajan sin acoplamiento al dominio: cambiar de `roles` a cualquier otra entidad no requiere tocar una sola línea de la vista que monta el formulario.

Capítulo 7

Desarrollo

7.1. Metodología

Para el desarrollo de este proyecto se ha optado por una metodología iterativa incremental, siguiendo los principios de desarrollo ágil y permitiendo la adaptación del desarrollo a las necesidades emergentes. Esta metodología se caracteriza por definir un número “N” de iteraciones en las que se va entregando por cada una de ellas una versión más completa del proyecto [6].

Asimismo, no se consideró correcta la elección de otras metodologías ágiles como “Scrum” ya que a pesar de ser un trabajo colaborativo, la línea que se desarrolla en este tfg era independiente.

La Metodología Iterativa Incremental encaja especialmente bien con el problema de un modelo de datos en evolución: cada incremento entrega una versión funcional del framework y, al cerrarlo, se analiza el estado del proyecto y las necesidades emergentes para planificar el siguiente. De este modo, la propia metodología absorbe la incertidumbre que motiva el trabajo.

7.2. Planificación

La planificación se organizó en seis fases, con una estimación total de **265 horas**, recogida en la tabla 7.1. Las fases de incremento se corresponden con las entregas funcionales del framework; la estimación se revisó al cierre de cada incremento, en coherencia con la planificación evolutiva acordada con la empresa.

La gestión del trabajo se apoyó en el repositorio Git del proyecto, con ramas de funcionalidad por incremento (por ejemplo, `Feature_Formly_Editor` para el Incremento 3) y revisiones de cierre con el tutor de empresa.

Cuadro 7.1: Planificación por fases y estimación de esfuerzo.

Fase	Descripción	Horas
1	Planificación: requisitos, arquitectura, entorno reproducible.	50
2	Incremento 1: API backend y modelado de datos.	50
3	Incremento 2: comunicación back-front (contrato).	60
4	Incremento 3: generación dinámica de formularios con Formly.	55
5	Incremento 4: generación automática de esquemas desde el ORM.	50
6	Documentación y presentación.	30
Total		265

7.3. Fases de desarrollo

7.3.1. Iteración 1

7.3.1.1. Análisis

El primer paso que se tomó para el desarrollo fue contactar tanto con el ITC, como con el JBVC para obtener una idea más concreta de las necesidades inmediatas que se requieren de este proyecto. Nótese que este desarrollo trata del framework que se va a utilizar para desarrollar la solución del JBVC, y no es la solución en sí. Sin embargo, se consideró necesaria la interacción con los Stakeholders para concretar la extensión de la abstracción requerida del framework.

La primera reunión con el JBVC se realizó el viernes 06 de febrero dentro del recinto del Jardín Botánico. En esta reunión se concretaron los detalles y requisitos funcionales que se requieren, así como las estructuras de datos ya existentes que requerirían de migración al nuevo sistema. Una vez establecidos los requisitos del proyecto el siguiente paso fue el de inspeccionar los repositorios que se habían suministrado para el desarrollo del proyecto.

Debido a que el proyecto ya estaba empezado, se requirió de un análisis previo del código ya escrito. Una vez observado el estado del desarrollo (Bastante temprano) se destinó la segunda semana de febrero entera a disponer de una base funcional y conectada entre los dos principales componentes en los que este TFG está fundamentado. Ya que se considera más eficiente en la industria el empaquetado de servidores/backends en Docker y a que los entornos de desarrollo que se trabajan en este proyecto son diversos (Windows, MacOS y Linux), se modificó ligeramente la base para que fuese posible ejecutarse en su totalidad con “docker compose”:

```
services:
```

```

ub_pg:
  image: postgres
  container_name: ub_pg
  environment:
    - POSTGRES_PASSWORD=pg_passwd # Contraseña de la base de datos
    - POSTGRES_DB=ub_db # Nombre de la base de datos
  ports:
    - "5433:5432" # Puerto externo e interno
  volumes:
    - ub_pg_data:/var/lib/postgresql # Volumen persistente
  restart: always # Reinicio automático

redis:
  image: redis:alpine
  container_name: redis
  ports:
    - "6379:6379" # Puerto de Redis
  restart: always

celery:
  image: python:3.13-slim
  container_name: celery_worker
  volumes:
    - ../uniback:/app # Código fuente de la aplicación
  working_dir: /app
  depends_on:
    - ub_pg # Depende de PostgreSQL
    - redis # Depende de Redis
  environment:
    - PYTHONPATH=/app/src
    - UNIBACK_DB_URL=postgresql://postgres:pg_passwd@ub_pg:5432/ub_db
    - UNIBACK_WORKER_BROKER_URL=redis://redis:6379/0
    - UNIBACK_WORKER_BACKEND_URL=redis://redis:6379/0
    - UNIBACK_WORKER_ENABLED=True
  # Integración con Celery (en desarrollo)
  # El comando real debe ajustarse a la ruta correcta de la app
  # command: celery -A uniback.worker worker --loglevel=info
  command: tail -f /dev/null # Mantiene el contenedor activo
  restart: on-failure

#Servicio añadido para permitir el "plug and play"
web:
  build:
    context: .
    dockerfile: Dockerfile
  image: uniback_web
  container_name: uniback_web
  ports:

```

```

    - "8000:8000" # Puerto del servidor web
volumes:
  - ./:/app
working_dir: /app
environment:
  - PYTHONPATH=/app/src
  - UNIBACK_DB_URL=postgres://postgres:pg_passwd@ub_pg:5432/ub_db
  - UNIBACK_REDIS_HOST=redis
command: uvicorn run_server:create_initialized_app --factory --host 0.0.0.0 --
  port 8000 --reload
depends_on:
  - ub_pg
  - redis

volumes:
  ub_pg_data: # Volumen para PostgreSQL

```

Algoritmo 7.1: Docker Compose de la aplicación UniBack

```

FROM python:3.13-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app

RUN apt-get update && apt-get install -y build-essential gcc libpq-dev && rm -rf /
  var/lib/apt/lists/*

# Copia los archivos del proyecto
COPY . /app

# Instala el proyecto en modo editable con dependencias de desarrollo
RUN pip install --upgrade pip setuptools wheel
RUN pip install -e ".[dev]"

ENV PYTHONPATH=/app/src

CMD ["uvicorn", "run_server:create_initialized_app", "--factory", "--host", "0.0.0.0",
  "--port", "8000"]

```

Algoritmo 7.2: Dockerfile de la aplicación UniBack

Una vez ejecutado y probado este método de despliegue con curl en dos equipos diferentes con diferentes sistemas operativos y diferentes arquitecturas de cpu, se paso a interconectar la librería NextGenDem-GUI-LIB con el nuevo despliegue de uniback.

7.3.1.2. Diseño e implementación del modelado de datos

Con el entorno reproducible en marcha, el Incremento 1 se centró en consolidar la capa de persistencia. El backend organiza los modelos ORM por dominio (`core`, `sysadmin`, `annotations`, `hierarchies`, `species`, `files`, `geographics`, `screens`). La pieza central es `FunctionalObject`: una clase base de la que heredan las entidades de dominio, que aporta identificador, `uuid`, propiedad (*ownership*), marcas de tiempo y atributos extensibles. Sobre ella se apoya un sistema de tipos de objeto (`ObjectType`) que asigna a cada clase un código numérico, lo que habilita el tratamiento polimórfico de las entidades.

El subsistema de autenticación y autorización emplea herencia polimórfica: `Identity` y `Role` heredan de `Authorizable`, de modo que las columnas comunes (`name`, `uuid`) residen en la tabla padre y solo las específicas (`email`, `can_login`, ...) en la tabla hija. El modelo de permisos se completa con `ACL`, `ACLExpression` y `ACLDetail`, junto con organizaciones, grupos y roles. El prefijo de tablas es configurable mediante `get_table_prefix()` (`ub_` por defecto), lo que evita colisiones cuando el framework convive con otras tablas en la misma base de datos.

7.3.1.3. Versionado e histórico con SQLAlchemy-Continuum

Para cumplir el requisito de histórico (RF-10) se integró SQLAlchemy-Continuum. La inicialización es delicada: `make_versioned` debe invocarse *antes* de definir cualquier modelo o crear la base ORM, por lo que se ubicó en el módulo `initializer`. Se habilitó el versionado nativo (`native_versioning`) y se configuró Alembic para soportarlo, de modo que las entidades núcleo (`FunctionalObject`, `Authorizable`, `Identity`, `FileSystemObject`, ...) mantienen automáticamente sus tablas de versiones. Las migraciones de esquema se gestionan con Alembic, con un conjunto de *revisions* versionadas en el repositorio.

7.3.1.4. Punto de entrada e inicialización

El framework expone una única función `initialize(config)` que normaliza la configuración con Pydantic Settings, prepara la base ORM y el gestor de sesiones, crea la aplicación FastAPI e inicializa de forma opcional Redis, el *worker* Celery y Firebase. Esta función es el contrato de uso del backend como librería: un proyecto consumidor solo necesita un diccionario de configuración y sus propios modelos, sin tocar el núcleo.

7.3.1.5. Decisiones de diseño y problemas encontrados

El problema más delicado del Incremento 1 fue el **orden de inicialización de SQLAlchemy-Continuum**. La extensión instrumenta los modelos en el momento de su definición, por lo que `make_versioned` debe ejecutarse antes de importar cualquier modelo o crear la base declarativa; ubicarlo en el módulo `initializer`, fuera de los modelos, resolvió un fallo intermitente de mapeo que solo se reproducía según el orden de importación. La segunda decisión

relevante fue el **prefijo de tablas configurable**: sin él, el framework colisionaría con tablas existentes al integrarse en un proyecto con su propia base de datos; resolverlo mediante `get_table_prefix()` mantiene la genericidad (RNF-01). Por último, la **herencia polimórfica** de `Authorizable` obligó a documentar que las consultas de identidades y roles requieren `join` con la tabla padre, algo no evidente que se trasladó a la documentación interna del proyecto.

7.3.2. Iteración 2: contrato de comunicación back–front

El Incremento 2 resuelve el acoplamiento histórico entre frontend y backend mediante un contrato uniforme, documentado en `docs/CONTRACT.md`.

7.3.2.1. Envoltorio universal de respuesta

Toda respuesta de la API — de éxito o de error — adopta la misma forma: un objeto con `content`, `count` e `issues`. El modelo se implementa con Pydantic (Algoritmo 7.3).

```
class ResponseEnvelope(BaseModel):
    content: Optional[Any] = None
    count: int = 0
    issues: List[Issue] = Field(default_factory=list)

    @classmethod
    def ok(cls, content=None, count=None, issues=None):
        if count is None:
            count = len(content) if isinstance(content, list) \
                else (1 if content is not None else 0)
        return cls(content=content, count=count, issues=issues or [])

    @classmethod
    def fail(cls, message, code=None, location=None):
        return cls(content=None, count=0,
            issues=[Issue.error(message, code=code,
                location=location)])
```

Algoritmo 7.3: Envoltorio de respuesta normalizado (`responses.py`)

El Algoritmo 7.4 ilustra la forma concreta del envoltorio en un caso de éxito y en un caso de error de validación.

```
{ "content": [ { "id": 1, "name": "admin" } ],
  "count": 1, "issues": [] }

{ "content": null, "count": 0,
  "issues": [ { "type": "ERROR", "code": "VALIDATION_ERROR",
    "message": "field required",
    "location": "/body/name" } ] }
```

Algoritmo 7.4: Envoltorio en éxito (arriba) y en error de validación (abajo)

Cada incidencia (**Issue**) tiene tipo (**INFO**, **WARNING**, **ERROR**), un código opcional y un **location** para apuntar al campo concreto en errores de validación. Unos manejadores de error globales capturan **HTTPException**, **RequestValidationError** y cualquier **Exception** no controlada y los traducen al mismo envoltorio, de modo que el frontend nunca recibe una forma de error distinta. La tabla 7.2 recoge el catálogo de códigos estándar.

Cuadro 7.2: Catálogo de códigos de error estándar del contrato.

Código	HTTP	Significado
BAD_REQUEST	400	Petición malformada
UNAUTHORIZED	401	Sesión inválida o ausente
FORBIDDEN	403	Sin permisos sobre el recurso
NOT_FOUND	404	Recurso inexistente
CONFLICT	409	Estado en conflicto
VALIDATION_ERROR	422	Validación Pydantic con <i>location</i>
INTERNAL_ERROR	500	Excepción no capturada

7.3.2.2. Fábricas CRUD y convención de endpoints

Para eliminar el código repetitivo por entidad se introdujeron las fábricas `make_simple_rest_crud` y `make_crudie_rest_crud`. Cada entidad expuesta a través de ellas ofrece automáticamente la misma batería de operaciones: listado con filtros, obtención por identificador, creación, actualización, borrado (*soft* o duro) y publicación de su JSON Schema en `/<entity>/schema.json`. La uniformidad de esta convención es lo que permite que el frontend trate cualquier entidad sin conocerla de antemano. La tabla 7.3 recoge la convención completa.

Cuadro 7.3: Convención de endpoints por entidad.

Método	Ruta	Propósito
GET	<code>/<e>/</code>	Lista (filtros como <i>query</i>)
GET	<code>/<e>/{id}</code>	Un objeto
POST	<code>/<e>/</code>	Crear (valida <code>create_schema</code>)
PUT	<code>/<e>/{id}</code>	Actualizar (valida <code>update_schema</code>)
DELETE	<code>/<e>/{id}</code>	Borrar (<code>soft_delete=true false</code>)
GET	<code>/<e>/schema.json</code>	JSON Schema enriquecido (ETag)

7.3.2.3. Descubrimiento

Dos endpoints de descubrimiento completan el contrato: `GET /api/sys/entities` lista todas las entidades CRUD detectadas en las rutas de la aplicación junto con sus capaci-

dades (*list/get/create/update/delete/schema*), y `GET /api/sys/discovery` lista los servicios externos detectados a través de Traefik. El descubrimiento es la pieza que hace el sistema *autodescriptivo*: el frontend puede preguntar al backend qué entidades existen en lugar de tenerlas codificadas.

7.3.2.4. Capa cliente genérica en ng2-gui

En el frontend se implementó una capa cliente reutilizable, exportada desde `ngt-gui`. `EntityClient` ofrece un cliente tipado por *path* de entidad; `SchemaService` cachea el esquema por entidad y lanza error si el envoltorio trae incidencias de tipo `ERROR`; `DiscoveryService` expone la lista de entidades como observable. Un `ApiErrorInterceptor` opcional convierte cualquier error HTTP en un `ApiResponse<null>` con `issues`; se diseñó *opt-in* para no romper el código heredado (RNF-06).

```
const roles = this.api.for<Role>('/roles');
roles.list().subscribe(res => /* ApiResponse<Role[]> */);
roles.create({ name: 'editor' }).subscribe(res => ...);
roles.schema<JSONSchema>().subscribe(res => ...);
```

Algoritmo 7.5: Uso de la capa cliente genérica desde un componente Angular

Junto a `EntityClient`, la capa incluye `SchemaService` (caché de esquemas por entidad y por *bundle*) y `DiscoveryService` (lista observable de entidades), como muestra el Algoritmo 7.6.

```
this.schemas.get('/roles').subscribe(s => /* JSON Schema */);
this.schemas.invalidate('/roles'); // forzar recarga
this.schemas.getBundle().subscribe(b => /* SchemaBundle */);

this.discovery.loadEntities().subscribe(l => /* EntityDescriptor[] */);
this.discovery.entities$.subscribe(l => /* observable */);
this.discovery.findEntity('roles');
```

Algoritmo 7.6: `SchemaService` y `DiscoveryService`

Es importante destacar una decisión de compatibilidad: el `BackendService` heredado, con sus ~272 métodos *hardcoded*, sigue funcionando igual que antes; la nueva capa convive con él y es la opción recomendada para entidades nuevas. La migración progresiva del servicio heredado queda fuera del alcance del Incremento 2.

7.3.2.5. Decisiones de diseño y problemas encontrados

La principal tensión del Incremento 2 fue la **compatibilidad hacia atrás**. El `BackendService` heredado estaba en producción y cualquier cambio en la forma de las respuestas lo habría roto. La solución fue diseñar la nueva capa como estrictamente *aditiva*: el envoltorio se introduce en los endpoints nuevos y en los manejadores de error globales, mientras que el servicio antiguo sigue devolviendo el cuerpo crudo. El `ApiErrorInterceptor` se dejó *opt-in* por el mismo motivo: registrarlo globalmente habría alterado el flujo de error de todo el código existente que captura `error: (err) =>...`. Otra decisión fue **normalizar también los errores**, no

solo los éxitos: capturar `HTTPException`, `RequestValidationError` y `Exception` en manejadores globales garantiza que el frontend jamás reciba dos formas distintas de fallo, lo que simplifica radicalmente el manejo de errores en el cliente.

7.3.3. Iteración 3: generación dinámica de formularios con Formly

El Incremento 3 es el núcleo de la aportación y está documentado en `docs/DYNAMIC_FORMS.md`. La meta es obtener *un formulario por entidad sin que nadie escriba HTML específico*, partiendo del esquema que el Incremento 2 ya publica.

7.3.3.1. Enriquecimiento del esquema en el backend

La conversión `sqlalchemy_to_pydantic` se amplió para inyectar, por columna, pistas de presentación en el `json_schema_extra` mediante la función `_ui_hints_for_column`. Estas pistas son heurísticas derivadas del tipo SQLAlchemy, resumidas en la tabla 7.4.

Cuadro 7.4: Heurísticas de enriquecimiento del esquema por tipo de columna.

Columna SQLAlchemy	Pista resultante
Boolean	<code>x-ui-widget: checkbox</code>
DateTime	<code>x-ui-widget: datepicker</code> (con hora)
Enum	<code>x-ui-widget: select</code>
String (>500)	<code>x-ui-widget: textarea</code>
JSON/JSONB	<code>x-ui-widget: json</code>
ForeignKey	<code>x-ui-widget: select-lazy-loading</code> y <code>x-options-endpoint</code>

El diseño respeta cualquier *override* manual: lo que el desarrollador declare en `column.info["ui"]` se aplana con prefijo `x-` y tiene prioridad sobre la heurística. Así, la automatización cubre el caso común sin impedir el ajuste fino. El Algoritmo 7.7 muestra el extracto real de la función que implementa estas reglas.

```
def _ui_hints_for_column(column) -> Dict[str, Any]:
    hints: Dict[str, Any] = {}
    col_type = column.type
    type_str = str(col_type).upper()

    if isinstance(col_type, sqltypes.Boolean):
        hints["x-ui-widget"] = "checkbox"
    elif isinstance(col_type, sqltypes.DateTime):
        hints["x-ui-widget"] = "datepicker"
        hints["x-formly-props"] = {"showTime": True}
    elif isinstance(col_type, sqltypes.Enum):
        hints["x-ui-widget"] = "select"
    elif isinstance(col_type, sqltypes.String):
        length = getattr(col_type, "length", None) or 0
        if length and length > 500:
            hints["x-ui-widget"] = "textarea"
```

```

elif "JSON" in type_str:
    hints["x-ui-widget"] = "json"

if column.foreign_keys:
    fk = next(iter(column.foreign_keys))
    ref_table = fk.column.table.name
    hints["x-ui-widget"] = "select-lazy-loading"
    hints["x-options-endpoint"] = f"/{ref_table}/"
    hints["x-options-label"] = "name"
    hints["x-options-value"] = fk.column.name or "id"

# Overrides manuales: column.info["ui"] y claves x-*
info = getattr(column, "info", None) or {}
ui_info = info.get("ui") if isinstance(info, dict) else None
if isinstance(ui_info, dict):
    for k, v in ui_info.items():
        key = k if k.startswith("x-") else f"x-{k}"
        hints[key] = v
return hints

```

Algoritmo 7.7: Heurística de pistas de UI por columna (`utils/common.py`)

7.3.3.2. Decisiones de diseño y problemas encontrados

La generación dinámica planteó varios retos cuya resolución conviene documentar. El primero fue **dónde inyectar las pistas**: situarlas en el `json_schema_extra` de Pydantic, y no en un canal paralelo, garantiza que viajen *dentro* del propio esquema y sobrevivan a cualquier serialización o cacheo. El segundo fue la **precedencia**: la heurística debía ser un valor por defecto, nunca una imposición, de ahí que los *overrides* de `column.info` se apliquen al final y ganen siempre. El tercero fue la **resolución de \$ref**: Pydantic emite referencias internas (`#/$defs/X`) para objetos anidados, lo que obligó a que el conversor del frontend las resolviese de forma recursiva antes de mapear tipos. Por último, se decidió **no soportar todavía oneOf/anyOf/allOf**: Pydantic los emite en escenarios de unión que el modelo actual no necesita, y resolverlos correctamente exige el discriminador polimórfico que se aborda en el Incremento 4. Acotar este límite de forma explícita evitó introducir complejidad especulativa.

7.3.3.3. Conversor de esquema a Formly en el frontend

La pieza clave del frontend es `schemaToFormly`, un conversor puro (sin dependencia de UI) que traduce un JSON Schema a un arreglo `FormlyFieldConfig[]`. Mapea los tipos primitivos a tipos Formly, resuelve referencias internas (`$ref`) de forma recursiva, traduce objetos anidados a `fieldGroup` y arreglos de objetos a `repeat`, e interpreta las pistas `x-*`: `x-formly-type`, `x-ui-widget`, `x-options-endpoint`, `x-placeholder`, `x-hide-when`, `x-required-when` y `x-disabled-when` se traducen a tipos, opciones y expresiones Formly. Además genera los validadores de Angular a partir de `minLength`, `maxLength`, `minimum`, `maximum`, `pattern` y `format`: `email`. La tabla 7.5 recoge el mapeo de tipos y la tabla 7.6 las extensiones `x-*` reconocidas.

Cuadro 7.5: Mapeo de JSON Schema a tipos Formly.

JSON Schema	Formly type
string	input
string + format: date-time	datepicker
string + enum	select
integer/number	input (number)
boolean	checkbox
array de object	repeat
object con properties	fieldGroup
\$ref interno	resuelve recursivamente

Cuadro 7.6: Extensiones x-* reconocidas por el conversor.

Keyword	Efecto
x-formly-type	Sobrescribe el <code>type</code> del campo
x-ui-widget	Alias semántico traducido a <code>type</code>
x-options-endpoint	Endpoint de opciones para <i>select lazy</i>
x-placeholder	<code>props.placeholder</code>
x-formly-props	Objeto fusionado en <code>props</code>
x-hide-when	Expresión $\rightarrow$ <code>expressions.hide</code>
x-required-when	Expresión $\rightarrow$ <code>props.required</code>
x-disabled-when	Expresión $\rightarrow$ <code>props.disabled</code>

7.3.3.4. Componente de formulario dinámico

El componente `<ngt-dynamic-form>` ofrece tres modos de uso, en orden creciente de dinamismo: a partir de un `FormlyFieldConfig[]` ya construido, a partir de un esquema en bruto, o — el caso más potente — a partir únicamente del *path* de la entidad, resolviendo el esquema con `SchemaService`.

```
<ngt-dynamic-form entityPath="/roles"
  [(model)]="model"
  (submitted)="save($event)">
</ngt-dynamic-form>
```

Algoritmo 7.8: Formulario generado solo a partir del path de la entidad

El componente admite modos `create`, `edit` y `view` (este último fuerza solo lectura), *overrides* por campo e incidencias del envoltorio para mostrarlas bajo el formulario, cerrando así el ciclo con el contrato del Incremento 2. La tabla 7.7 resume sus entradas.

Cuadro 7.7: Entradas del componente `<ngt-dynamic-form>`.

Input	Notas
<code>entityPath</code>	Se resuelve con <code>SchemaService</code>
<code>schema</code>	Esquema en bruto
<code>fields</code>	<code>FormlyFieldConfig[]</code> (omite el conversor)
<code>model</code>	Modelo bidireccional
<code>mode</code>	<code>create edit view</code>
<code>overrides</code>	<i>Deep-merge</i> por clave
<code>issues</code>	Incidencias a mostrar bajo el formulario

Sus salidas son `modelChange`, `submitted` y `fieldsReady`, lo que permite que la vista contenedora reaccione al envío y al modelo sin conocer la estructura concreta de la entidad. Cabe señalar una limitación conocida y documentada: el conversor procesa propiedades de primer nivel y sus hijos inmediatos, y los *overrides* son por clave de primer nivel; el soporte de *overrides* por *path* queda registrado como mejora futura.

7.3.4. Iteración 4: generación automática de esquemas

El Incremento 4 formaliza la extracción de esquemas desde el ORM y los publica en un *bundle* único versionado, documentado en `docs/SCHEMA_GENERATION.md`. La iteración quedó **cerrada**: el registro de entidades, el endpoint `/api/sys/schemas`, el versionado por ETag, el reflejo del polimorfismo y de las relaciones del ORM, y sus pruebas están en su sitio.

Cada llamada a una fábrica CRUD registra la entidad en un *registry* global (`EntityRegistration`: nombre, *path*, factoría, clase ORM, esquema Pydantic, etiquetas). El Algoritmo 7.9 muestra la estructura de registro.

```
@dataclass
class EntityRegistration:
    name: str          # "roles"
    path: str          # "/roles"
    factory: str        # "simple" | "crudie"
    orm_class: Type | None
    pydantic_schema: Type[BaseModel] | None
    tags: list[str]
```

Algoritmo 7.9: Registro de entidad (`schema_registry.py`)

A partir de él, `build_schema_bundle` construye un documento con el esquema enriquecido de todas las entidades y `compute_etag` calcula un hash SHA-256 truncado, encapsulado como validador débil W/"...". El Algoritmo 7.10 muestra la forma del *bundle*.

```
{ "$schema": "https://json-schema.org/draft/2020-12/schema",
  "version": "9f3a1b2c4d5e6f70",
  "generated_at": "2026-05-11T10:38:02Z",
  "entities": {
    "roles": { "type": "object", "properties": { } },
    "identities": { "type": "object", "properties": { } }
  } }
```

Algoritmo 7.10: Documento del bundle de esquemas

El bundle incluye `version` (el ETag, estable entre llamadas con la misma BD/ORM) y `generated_at` (metadato que no participa en el hash). Tanto el *bundle* como cada `/<entity>/schema.json` responden **HTTP 304** ante un `If-None-Match` coincidente, lo que habilita *caching* agresivo en navegador y proxies (RNF-05).

El valor de esta pieza para la reusabilidad es directo: un script externo puede pedir el *bundle* una sola vez y generar interfaces TypeScript, formularios precompilados, validadores o documentación; y un microservicio puede espejar el contrato de otro sin acoplarse a su ORM.

7.3.4.1. Polimorfismo y relaciones del ORM

El cierre del incremento añadió las dos piezas que faltaban, ambas inyectadas en `get_enriched_json_schema` para que afecten por igual a `/<entity>/schema.json` y al *bundle* agregado. La primera es el **polimorfismo**: si la clase ORM es la raíz de una jerarquía (`polymorphic_on` definido y sin mapper padre), el esquema emite un `oneOf` con el esquema enriquecido de cada subclase y un `x-discriminator` (`propertyName + mapping valor→subclase`) deducido de `object_type_id`. La segunda es el reflejo de los `relationship()` del ORM como un array `x-relationships` (nombre, destino, dirección, si es colección, columnas FK locales y *end-point* si la entidad está registrada), que complementa el `x-foreign-key` por columna ya existente.

7.3.4.2. Decisiones de diseño y problemas encontrados

La decisión más relevante fue **incrustar las variantes completas** en el `oneOf` en lugar de referenciarlas con `$ref` a `#$defs`: dentro del *bundle* las entidades cuelgan de `entities.<nombre>`, por lo que un `#$defs` no resolvería desde la raíz del documento; incrustarlas mantiene el *bundle* autocontenido. La segunda fue **conservar las `properties base`** junto al `oneOf`: así un consumidor que aún no entienda `oneOf` —como el conversor a Formly actual— sigue viendo las columnas comunes sin romperse, lo que permitió acotar el alcance en el frontend a no desestabilizar el conversor (sin un selector de tipo todavía). La tercera fue el **determinismo**: las variantes se ordenan por valor de discriminador y las relaciones por nombre, de modo que el `version/ETag` del *bundle* permanece estable entre generaciones idénticas, requisito de *caching* (RNF-05). Un problema colateral surgido al ejecutar la batería de pruebas en la base SQLite en memoria fue que el tipo PostgreSQL `TSVECTOR` no es compilable en SQLite; se resolvió, en línea con los tipos ya independientes de plataforma (`GUID`, `JSONB`), con un `@compiles` que lo representa como `TEXT` fuera de PostgreSQL, sin alterar el comportamiento nativo.

7.4. Pruebas

Cada incremento se acompañó de pruebas automatizadas, en coherencia con el requisito de fiabilidad (RNF-04).

7.4.1. Backend

Las pruebas del backend se ejecutan con `pytest`. Destacan: `test_envelope.py` (forma del envoltorio, 404, validación, `/sys/entities`, `schema.json`), `test_auth.py` (autenticación), `test_form_schema.py` (presencia de `x-ui-widget` en columnas *datetime* y `x-options-endpoint` en *foreign keys*, más *unit tests* de `_ui_hints_for_column`) y `test_schema_bundle.py` (versión estable, *round-trip* de ETag con 304, sensibilidad del hash al contenido).

7.4.2. Frontend

En el frontend, `schema-to-formly.spec.ts` cubre los tipos primitivos, `required`, *enum/select*, los *overrides* `x-formly-type` y `x-ui-widget`, `x-options-endpoint`, `fieldGroup`, `fieldArray`, resolución de `$ref`, las expresiones (`x-hide-when`) y el modo `view`, incluyendo el caso de una raíz polimórfica con `oneOf/x-discriminator`. El flujo extremo a extremo — descubrimiento, obtención de esquema, generación del formulario y envío mediante `EntityClient`, mostrando las incidencias del envoltorio — queda encapsulado en el componente reutilizable `<ngt-dynamic-form>`, que compone esas piezas sin acoplarse al dominio.

7.4.3. Resumen de cobertura

La tabla 7.8 relaciona cada incremento con sus pruebas automatizadas y el aspecto que verifican.

Cuadro 7.8: Pruebas automatizadas por incremento.

Inc.	Suite	Qué verifica
1	<code>test_initialization.py</code>	Inicialización de la base ORM y la app.
2	<code>test_envelope.py</code>	Forma del envoltorio, 404, validación, <code>/sys/entities, schema.json</code> .
2	<code>test_auth.py</code>	Autenticación y modo local de desarrollo.
3	<code>test_form_schema.py</code>	Pistas <code>x-*</code> en columnas <code>datetime</code> y FK; <code>unit</code> de <code>_ui_hints_for_column</code> .
3	<code>schema-to-formly.spec.ts</code>	Tipos, <i>enum</i> , <i>overrides</i> , <code>\$ref</code> , expresiones, modo <i>view</i> .
4	<code>test_schema_bundle.py</code>	Versión estable, ETag <i>round-trip</i> (304), sensibilidad del hash, polimorfismo (<code>oneOf+ x-discriminator</code>) y <code>x-relationships</code> .
4	<code>schema-to-formly.spec.ts</code>	Robustez del conversor ante raíz polimórfica (no rompe, no filtra variantes).

La estrategia de pruebas combina *unit tests* de las piezas puras (la heurística de pistas y el conversor a Formly) con pruebas de integración de la API (envoltorio, descubrimiento, esquema, bundle versionado y polimorfismo) ejecutadas sobre una base de datos en memoria. Esta combinación cubre los puntos donde un cambio del modelo de datos podría romper la cadena de generación, que es exactamente el riesgo que el framework pretende neutralizar.

Capítulo 8

Resultados

Presentación de los resultados del trabajo. Repercusión.

8.1. Resultados por incremento

El desarrollo produjo un framework funcional cuyos cuatro incrementos están consolidados y respaldados por pruebas automatizadas.

8.1.1. Incremento 1: API backend y modelado (consolidado)

Se dispone de un entorno de despliegue reproducible con `docker compose`, probado en sistemas operativos y arquitecturas de CPU distintas, y de un núcleo de modelos ORM organizado por dominio, con herencia polimórfica para autenticación/autorización, prefijo de tablas configurable, versionado con SQLAlchemy-Continuum y migraciones con Alembic. El punto de entrada `initialize(config)` permite usar el backend como librería.

8.1.2. Incremento 2: contrato back–front (consolidado)

El contrato uniforme está implementado y documentado: envoltorio de respuesta normalizado, manejadores de error globales, catálogo de códigos estándar, fábricas CRUD, endpoints de descubrimiento y una capa cliente genérica tipada en Angular que convive con el código heredado sin romperlo. Las pruebas `test_envelope.py` y `test_auth.py` verifican la forma del envoltorio, los errores y el descubrimiento.

8.1.3. Incremento 3: formularios dinámicos (consolidado)

La cadena esquema→formulario funciona extremo a extremo: el backend enriquece el esquema con pistas `x-*` a partir de los tipos SQLAlchemy y el frontend genera, sin HTML específico, un formulario Formly completo con validadores y expresiones condicionales. El componente reutilizable `<ngt-dynamic-form>` encapsula el flujo completo (descubrimiento → esquema → formulario → envío) y las pruebas `test_form_schema.py` y `schema-to-formly.spec.ts` cubren los casos relevantes.

8.1.4. Incremento 4: bundle de esquemas (consolidado)

El *registry* de entidades, el endpoint `/api/sys/schemas`, el versionado por ETag y sus pruebas (`test_schema_bundle.py`) están implementados y operativos. El incremento se completó añadiendo las dos piezas que quedaban pendientes: el reflejo del **polimorfismo** de `FunctionalObject` y subclases mediante `oneOf` con un `x-discriminator` sobre `object_type_id`, y el de los `relationship()` del ORM como extensión `x-relationships`. Ambas viajan dentro del propio esquema enriquecido, son deterministas (no desestabilizan el ETag del *bundle*) y conservan las `properties` base, de modo que un consumidor que no entienda `oneOf` sigue funcionando. Las pruebas verifican que la entidad raíz expone `oneOf+x-discriminator` con sus subclases, que una subclase no lo emite, y que `x-relationships` refleja las relaciones resueltas contra el *registry*.

8.2. Valoración global

La tabla 8.2 resume el estado de los objetivos específicos. El resultado más significativo es cualitativo: exponer y editar una entidad nueva pasa de requerir un router, esquemas y formularios escritos a mano a *declarar el modelo ORM* y dejar que el framework derive el resto. Esto valida la hipótesis de partida sobre un modelo de datos en evolución.

8.2.1. Comparativa antes/después

La forma más clara de valorar el impacto es comparar el trabajo necesario para poner en producción una entidad nueva, editable desde el frontend, antes y después del framework (tabla 8.1).

El cambio cualitativo es el relevante: el esfuerzo deja de crecer con el número de entidades y con la frecuencia de cambios del modelo, que era precisamente el problema de partida. La verificación de que la cadena no se rompe ante un cambio del modelo queda garantizada por las pruebas de la heurística de pistas y del conversor.

Cuadro 8.1: Trabajo para exponer y editar una entidad nueva.

Tarea	Antes	Después
Endpoints CRUD	Router escrito a mano	Una fábrica CRUD
Validación	Esquemas Pydantic a mano	Derivada del ORM
Contrato/errores	<i>Ad hoc</i> por endpoint	Envoltorio uniforme
Cliente frontend	Método dedicado en <code>BackendService</code>	<code>EntityClient.for(path)</code>
Formulario	HTML + lógica por campo	<code><ngt-dynamic-form entityPath></code>
Coste ante cambio del modelo	Propagación manual a 3 capas	Automática desde el ORM

Cuadro 8.2: Grado de consecución de los objetivos específicos.

Objetivo específico	Estado
Crear una API modular	Completado
Diseñar y modelar los esquemas de datos	Completado
Comunicación estandarizada back-front	Completado
Generador automático de formularios (Formly)	Completado
Generación automática de esquemas desde el ORM	Completado

Capítulo 9

Conclusiones y trabajo futuro

Este es un CAPÍTULO OBLIGATORIO.

9.1. Conclusiones

Valoración de resultados, grado de consecución de los objetivos.

El trabajo partía de un problema concreto: el alto coste de mantener sincronizados backend, comunicación e interfaz ante un modelo de datos en evolución. La solución construida demuestra que ese coste puede reducirse de forma drástica si el modelo ORM se convierte en la única fuente de verdad y el resto del sistema se deriva de él mediante un contrato uniforme y una cadena de generación automática.

Los cinco objetivos específicos planteados se han completado y consolidado con pruebas (API modular, modelado de datos, comunicación estandarizada, generación dinámica de formularios y generación automática de esquemas desde el ORM, esta última incluyendo el polimorfismo y las relaciones del modelo). El grado de consecución de los objetivos generales es, por tanto, alto: existe un framework Backend/Frontend reutilizable, con una arquitectura desacoplada del dominio y adaptada de forma evolutiva a las necesidades reales del ITC y del JBVC. La metodología iterativa incremental resultó especialmente adecuada, pues su planificación evolutiva absorbió la incertidumbre inicial del backend y permitió cerrar cada incremento con una versión funcional y verificada.

9.2. Trabajo futuro

Líneas de trabajo futuro, aspectos pendientes, posibles extensiones.

Las líneas de continuación más relevantes son:

- **Codegen:** aprovechar el *bundle* versionado —ya con polimorfismo (**oneOf**+**x-discriminator**) y relaciones (**x-relationships**)— para generar automáticamente interfaces TypeScript, validadores y documentación en proyectos consumidores.
- **Migración del BackendService heredado** para que delegue internamente en **EntityClient**, retirando progresivamente los métodos *hardcoded*.
- **Soporte de oneOf/anyOf/allOf** en el conversor a Formly, y *overrides* por *path* (no solo por clave de primer nivel).
- **Generación automática de vistas de listado/tabla** con la misma filosofía guiada por esquema que ya se aplica a los formularios.

9.3. Uso de la IA

Indicar el uso que se ha hecho de la IA tanto en la elaboración de este documento como en el desarrollo del TFG.

Durante el desarrollo del TFG se ha empleado un asistente de inteligencia artificial como apoyo, siempre bajo revisión y validación del autor. En el **desarrollo del software** se utilizó para acelerar tareas repetitivas, explorar el código heredado, sugerir refactorizaciones y depurar; las decisiones de diseño, la integración y la verificación finales fueron propias. En la **elaboración de esta memoria** se utilizó como ayuda de redacción y estructuración a partir del código y la documentación reales del proyecto, revisando y corrigiendo el autor todo el contenido para garantizar su exactitud técnica y su autoría.

EXTENSIÓN DE LA MEMORIA:

- GII, GIFM, DGII-ADE (parte de GII): entre 50 y 100 páginas
- GCID: entre 75 y 100 páginas

Capítulo 10

Aspectos económicos y temporales

10.1. Planificación temporal

El proyecto se planificó en 265 horas distribuidas en seis fases (tabla 7.1). La naturaleza evolutiva de la planificación, acordada con la empresa, implicó revisar la estimación al cierre de cada incremento. La tabla 10.1 recoge la desviación entre lo estimado y lo realmente invertido, así como el resultado de cada fase.

Cuadro 10.1: Estimación frente a esfuerzo real y estado por fase.

Fase	Est.	Real	Estado
Planificación y entorno	50	48	Completada
Inc. 1: API y modelado	50	56	Completada
Inc. 2: contrato back-front	60	62	Completada
Inc. 3: formularios Formly	55	58	Completada
Inc. 4: esquemas desde ORM	50	34	Parcial
Documentación y presentación	30	30	En curso
Total	265	288	

La desviación se concentra en los incrementos 1–3, que requirieron más esfuerzo del estimado por la curva de aprendizaje del código heredado; el Incremento 4 se completó incluyendo el polimorfismo y las relaciones del ORM en el *bundle* de esquemas.

10.2. Presupuesto

El presupuesto se calcula imputando el esfuerzo a un coste de ingeniería de software junior y añadiendo los costes de equipamiento y de licencias. Todo el *software* empleado es de código abierto, por lo que el coste de licencias es nulo. La tabla 10.2 resume el cálculo.

Cuadro 10.2: Presupuesto estimado del proyecto.

Concepto	Cant.	Coste ud.	Total
Mano de obra (ingeniería)	288 h	18 €/h	5.184 €
Amortización equipo (4 meses)	1	60 €/mes	240 €
Licencias de software (FOSS)	—	0 €	0 €
Infraestructura/servicios	4 meses	0 €	0 €
Subtotal			5.424 €
Costes indirectos (10 %)			542 €
Total estimado			5.966 €

El retorno de esta inversión no se mide en el propio TFG sino en los proyectos que lo reutilicen: al eliminar el código repetitivo de exposición, validación y formularios, el ahorro esperado por entidad nueva es de varias horas de desarrollo y mantenimiento, lo que amortiza el coste del framework ya en el primer proyecto que lo adopte.

Capítulo 11

Normativa y legislación

11.1. Protección de datos

El framework no almacena por sí mismo datos personales: es una herramienta de construcción. No obstante, las aplicaciones que se construyan con él pueden gestionar datos de carácter personal (identidades de usuario, autoría de registros), por lo que el diseño se ha realizado teniendo presente el Reglamento General de Protección de Datos (RGPD, Reglamento (UE) 2016/679) y la Ley Orgánica 3/2018 de Protección de Datos Personales y Garantía de los Derechos Digitales (LOPDGDD).

Las decisiones de diseño relevantes para este cumplimiento son: la autenticación delega la verificación de credenciales en un proveedor externo (Firebase) mediante verificación de *token*, evitando almacenar contraseñas; el modo de *login* local existe únicamente para desarrollo y no debe habilitarse en producción (RNF-07); el versionado con SQLAlchemy-Continuum mantiene un histórico de cambios que, en un despliegue real, debe acompañarse de políticas de retención y del derecho de supresión; y el modelo de autorización basado en roles y ACL permite implementar el principio de mínimo privilegio sobre los datos.

11.2. Licencias de software

El backend *uniback* se distribuye bajo licencia MIT, lo que permite su reutilización en proyectos de la empresa sin fricción legal. Las dependencias empleadas (FastAPI, SQLAlchemy, Pydantic, Alembic, SQLAlchemy-Continuum, Angular, Formly, PostgreSQL, Redis, Docker) son software libre con licencias permisivas (MIT, BSD, Apache 2.0, PostgreSQL License) compatibles entre sí y con un uso comercial. No se ha incorporado código con licencias copyleft fuertes que pudieran imponer restricciones a la distribución del framework.

11.3. Propiedad intelectual

El framework se desarrolla en colaboración con el ITC. La titularidad y las condiciones de explotación se rigen por el acuerdo de colaboración entre la universidad, el estudiante y la empresa, y se reflejan en la correspondiente solicitud de defensa del trabajo.

Capítulo 12

Manual de software

Este capítulo recoge los extractos de código más relevantes y las instrucciones de uso del framework. El código fuente completo está disponible en los repositorios del proyecto: el backend *uniback* y el frontend *gui-components* (librería `ngt-gui`).

12.1. Despliegue

El sistema completo se levanta con un único comando gracias a la configuración `docker compose` descrita en el capítulo de Desarrollo (Algoritmos 7.1 y 7.2):

```
docker compose up --build
# API: http://localhost:8000
# PostgreSQL: localhost:5433 (db ub_db, user postgres)
```

Algoritmo 12.1: Levantar el entorno completo

12.2. Uso del backend como librería

Un proyecto consumidor inicializa el framework con un diccionario de configuración y añade sus propios modelos sobre la base ORM devuelta, sin modificar el núcleo.

```
from uniback import initialize

config = {
    "database": {"url": "postgresql://postgres:pg_passwd@ub_pg:5432/ub_db"},
    "api": {"title": "Mi_API", "version": "1.0.0"},
}
orm_base, session_factory, app = initialize(config)

from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import String
```

```
class MiEntidad(orm_base):
    __tablename__ = "mi_entidad"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100))
```

Algoritmo 12.2: Inicialización y modelo propio del consumidor

12.3. Exposición de una entidad y consumo del contrato

Tras exponer la entidad con una fábrica CRUD, queda disponible toda la convención de endpoints (listar, obtener, crear, actualizar, borrar y `schema.json`). El Algoritmo 12.3 muestra el ciclo de prueba rápido con curl usando el *login* local de desarrollo.

```
# Login local de desarrollo (solo dev)
curl -c cookies.txt -X PUT "http://localhost:8000/api/authn?user=admin"
# Esquema enriquecido de la entidad
curl -b cookies.txt http://localhost:8000/roles/schema.json
# Listado (envoltorio: content / count / issues)
curl -b cookies.txt http://localhost:8000/roles/
# Bundle versionado de todos los esquemas
curl -b cookies.txt -i http://localhost:8000/api/sys/schemas
```

Algoritmo 12.3: Prueba rápida del contrato con curl

12.4. Generación del formulario en el frontend

En la aplicación Angular, registrar el módulo de tipos Formly y declarar el componente dinámico es suficiente para obtener un formulario completo a partir del *path* de la entidad.

```
// app.module: cargar FormlyComponentsModule (tipos ng-zorro + custom)
@Component({
  template: `
    <ngt-dynamic-form entityPath="/roles"
      [(model)]="model" mode="create"
      (submitted)="save($event)">
    </ngt-dynamic-form>`
})
export class RoleCreatePage {
  model: Record<string, unknown> = {};
  save(value: unknown) { /* EntityClient.create() ya invocado */ }
}
```

Algoritmo 12.4: Formulario dinámico en una vista Angular

12.5. Cómo escribir un plugin

La forma recomendada de añadir un dominio nuevo no es tocar el núcleo, sino escribir un plugin (sección 6.3). Un plugin es un paquete Python dentro de `src/plugins/`; el `PluginManager` lo descubre solo al arrancar buscando `<paquete>/plugin.py` y, dentro, una subclase de `UnibackPlugin`. No hay que registrarlo en ningún sitio. La estructura mínima es:

```
src/plugins/plants/
|-- __init__.py # docstring del paquete
|-- models.py # modelo SQLAlchemy del dominio
'-- plugin.py # la clase UnibackPlugin con los hooks
```

Algoritmo 12.5: Estructura mínima de un plugin

El modelo hereda de `FunctionalObject` para reaprovechar `id/uuid/name`, propiedad automática, marcas de tiempo, borrado lógico y polimorfismo. El tipo de cada columna SQLAlchemy determina el *widget* del formulario (`Date` → *datepicker*, `Enum` → *select*, `Float` → *numérico...*), y `info={“ui”: ...}` permite forzar un *override* manual.

```
class Plant(FunctionalObject):
    __tablename__ = f"{get_table_prefix()}plants_plants"
    __mapper_args__ = {"polymorphic_identity": data_object_type_id["plant"]}
    id: Mapped[int] = mapped_column(
        BigInteger,
        ForeignKey(f"{get_table_prefix()}functional_objects.id"),
        primary_key=True,
    )
    species: Mapped[str | None] = mapped_column(String(200))
    latitude: Mapped[float | None] = mapped_column(Float) # -> input numerico
    longitude: Mapped[float | None] = mapped_column(Float)
    planted_on: Mapped[date | None] = mapped_column(Date) # -> datepicker
    health: Mapped[PlantHealth | None] = mapped_column( # Enum -> select
        SAEnum(PlantHealth, name="plant_health"))
    notes: Mapped[str | None] = mapped_column(
        String(2000),
        info={"ui": {"ui-widget": "textarea"}}, # override manual
    )
```

Algoritmo 12.6: Modelo de dominio de un plugin (`plants/models.py`)

La clase del plugin engancha el modelo y su CRUD a los *hooks* (Algoritmo 6.3). Con eso, y sin escribir nada más, la entidad aparece en `/api/sys/entities`, se incluye en el *bundle* versionado, publica su `schema.json` y queda lista para que `<ngt-dynamic-form>` genere su formulario en el frontend. Lo único que se escribe a mano en la interfaz es lo propio de la app (en el ejemplo de las plantas, el mapa). El recorrido completo está documentado en `docs/TUTORIAL_APP_PLANTAS.md`.

12.6. Estructura de los repositorios

El backend sigue la estructura `src/uniback/{persistence, services, api, authorization, config}` con pruebas en `tests/` y documentación del contrato en `docs/` (`CONTRACT.md`, `DYNAMIC_FORMS.md`, `SCHEMA_GENERATION.md`). El frontend sigue `projects/ngt-gui/src/lib/{core, services, components, models}` con la API pública re-exportada en `public-api.ts`. Esta documentación interna es la referencia de uso para los desarrolladores que adopten el framework.

Capítulo 13

Otros capítulos y anexos

El documento debe terminar con las **referencias bibliográficas** (SECCIÓN OBLIGATORIA). Después pueden incluirse **anexos** de forma opcional.

13.1. Ejemplos de capítulos opcionales

Dependiendo del tipo de trabajo, se podrían incluir, en el orden que corresponda, capítulos adicionales como los siguientes:

REQUISITOS

NORMATIVA Y LEGISLACIÓN incluir la legislación vigente que afecte al TFT (ley de protección de datos, leyes sobre seguridad, ...)

ASPECTOS ECONÓMICOS Y TEMPORALES

DISEÑO

RESULTADOS EXPERIMENTALES

MANUAL DE USUARIO Y SOFTWARE deberán incluirse obligatoriamente en la memoria los extractos más relevantes del código desarrollado. Siempre que sea posible, deberá proporcionarse acceso a un repositorio software.

Capítulo 14

informacion academica

EXTENSIÓN DE LA MEMORIA:

- GII, GIFM, DGII-ADE (parte de GII): entre 50 y 100 páginas
- GCID: entre 75 y 100 páginas

14.1. Ejemplo de sección - estilo de citas

Así se cita una referencia bibliográfica [2], o la tabla 14.1 o la figura 14.1.

14.1.1. Ejemplo de sub-sección

No deben utilizarse niveles adicionales (sub-sub-secciones).

Si hay texto que se ha copiado literalmente de algún sitio, hay que entrecomillarlo “En un lugar de La Mancha, de cuyo nombre no quiero acordarme” y **citar** el origen (Andrés Iniesta).

Cuadro 14.1: Ejemplo de tabla blablabla.

	Gallery				
	Floor	0	1	2	3
Probe	0	98.5 / 98.9	92.2 / 92.2	77.4 / 77.4	80.7 / 81.9
	1	91.5 / 92.2	98.3 / 98.6	83.5 / 85.2	80.7 / 81.1
	2	69.3 / 69.3 / 69.7	83.5 / 84.4	94.8 / 97.8	75.8 / 80.2
	2	69.3 / 69.3 / 69.7	83.5 / 84.4	94.8 / 97.8	75.8 / 80.2
	2	69.3 / 69.3 / 69.7	83.5 / 84.4	94.8 / 97.8	75.8 / 80.2
	2	69.3 / 69.3 / 69.7	83.5 / 84.4 / 84.4	94.8 / 97.8	75.8 / 80.2
	3	66.5 / 67.4 / 68.3	72.7 / 72.7 / 73.0	76.3 / 80.1	97.8 / 99.4

Todas las figuras/ilustraciones o cuadros/tablas deben estar comentadas en el texto. No es adecuado usarlas como parte de la redacción, lo que se muestra en el pie debe ser un resumen de lo que ya se describe en el documento. Tampoco deben emplearse referencias del tipo, “en la figura que se muestra a continuación” o “en la tabla anterior”, hay que usar referencias indexadas que no dependan de la posición. Un ejemplo de esto sería: “La ilustración 14.1 muestra el logo de la Escuela”.

14.2. Ejemplo de sección - organización del documento

Al final de la introducción suele describirse la estructura del documento, indicando los capítulos que vienen a continuación y una frase corta describiendo su contenido. “El resto del documento está integrado por los capítulos correspondientes al análisis de requisitos del trabajo, el capítulo dedicado al diseño de la aplicación, ... y finalmente las conclusiones que se han derivado del trabajo.”

En el caso del **doble grado**, lo ideal es dividir la memoria en dos partes separadas, una para cada grado, después de la introducción. También deberá incluirse una guía de lectura, indicando qué capítulos/secciones son comunes y cuáles exclusivos de cada grado,



Ilustración 14.1: Logo vertical de la EII y la UPGC [indicar crédito si no es una figura propia. Internet o Google como buscador no es una fuente, habría que citar el origen concreto]

Algoritmo 1 Pseudocódigo con comentarios.

Require: $n \geq 0$ **Ensure:** $y = x^n$ $y \leftarrow 1$ $X \leftarrow x$ $N \leftarrow n$ **while** $N \neq 0$ **do** **if** N is even **then** $X \leftarrow X \times X$ $N \leftarrow \frac{N}{2}$ **else if** N is odd **then** $y \leftarrow y \times X$ $N \leftarrow N - 1$ **end if****end while**

▷ This is a comment

para facilitar la corrección de los correspondientes tribunales.

La estructura que se propone a continuación es simplemente orientativa, aunque sí deben incluirse de alguna manera las **partes obligatorias**, que son los capítulos de **motivación y objetivos**, **competencias**, **aportaciones y alineamiento ODS**, **desarrollo**, **conclusiones y bibliografía**.

Capítulo 15

Anexos

15.1. Anexo A. Referencia de endpoints del sistema

La tabla 15.1 recoge los endpoints transversales del framework (no ligados a una entidad concreta) que conforman la superficie de descubrimiento y esquema del contrato.

Cuadro 15.1: Endpoints transversales del contrato.

Método	Ruta	Propósito
GET	/api/sys/entities	Entidades CRUD y sus capacidades
GET	/api/sys/discovery	Servicios externos (Traefik)
GET	/api/sys/schemas	Bundle versionado de esquemas (ETag)
GET	/<entity>/schema.json	Esquema enriquecido de la entidad (ETag, 304)
PUT	/api/authn?user=...	Login local de desarrollo
GET	/health	Sonda de estado del servicio

15.2. Anexo B. Referencia de configuración

La inicialización acepta las secciones de configuración resumidas en la tabla 15.2, validadas por Pydantic Settings y sobrescribibles mediante variables de entorno con prefijo UNIBACK_.

Cuadro 15.2: Secciones de configuración del inicializador.

Sección	Claves principales
database	url, echo, pool_size
api	title, version, debug, cors_origins
auth	secret_key, algorithm, firebase_credentials_path
worker	enabled, broker_url, backend_url

15.3. Anexo C. Acceso al código fuente

El código completo del framework reside en dos repositorios: el backend *uniback* (Python, paquete instalable con `pip install -e ".[dev]"`) y el frontend *gui-components*, que contiene la librería publicable *ngt-gui*, en la que el componente `<ngt-dynamic-form>` integra el contrato extremo a extremo. La documentación interna del contrato y de los incrementos se encuentra en la carpeta `docs/` del backend (`CONTRACT.md`, `DYNAMIC_FORMS.md`, `SCHEMA_GENERATION.md`), y constituye la guía de referencia para los desarrolladores que reutilicen el framework.

Bibliografía

- [1] Bayer, M. (2024). *SQLAlchemy 2.x Documentation*. <https://docs.sqlalchemy.org>.
- [2] Castrillón, M., Déniz, O., Hernández, D., and Lorenzo, J. (2011). A comparison of face and facial feature detectors based on the violajones general object detection framework. *Machine Vision and Applications*, 22(3):481–494.
- [3] Formly (2024). *Formly — Dynamic (JSON powered) forms in Angular*. <https://formly.dev>.
- [4] Google (2024). *Angular Documentation*. <https://angular.dev>.
- [5] JSON Schema Org (2020). *JSON Schema Specification (Draft 2020-12)*. <https://json-schema.org>.
- [6] Larman, C. (2004). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*. Prentice Hall, 3rd edition.
- [7] Ramírez, S. (2024). *FastAPI — Documentación oficial*. <https://fastapi.tiangolo.com>.
- [8] Raymond, E. S. (1999). *The Cathedral & the Bazaar: Musings on Linux and Open Source by an Accidental Revolutionary*. O'Reilly Media.

